



CHRONIQO DESIGN & IMPLEMENTERING

SW7BAC-01 BACHELORPROJEKT

BILAG 6

Elev:

Mads Dittmann Villadsen

202209869 / AU715368

Vejleder:

Tommy Bjerre Nielsen

tbn@ece.au.dk

MAJ 2026

Indholdsfortegnelse

Figurliste	iii
Tabelliste	iv
Kodeudsnitsliste	v
1. Indledning og formål.....	1
2. UI/UX-design - implementeringsdetaljer.....	2
2.1. Designsystem og CSS-variabler (globals.css)	3
2.1.1. Farvepalet og temavariabler (Light-/Dark Mode).....	3
2.1.2. Typografisk skala (Poppins, Inter og Syne)	7
2.1.3. Temainitialisering (forebyggelse af FOUC)	8
2.1.4. Animationsbibliotek	10
2.2. Dagsformsindikatoren - komponentimplementering	11
2.2.1. DailyStatusContext - tilstandsstyring og distribution	12
2.2.2. DailyStatusInterceptor - dialogvinduelogik og registreringsflow	15
2.2.3. DailyStatusSlider - brugergrænsefladekomponent	19
2.3. Det adaptive UI (Low Energy Mode) - implementeringsdetaljer	22
2.3.1. Tilstandstransition - hvornår aktiveres Low Energy Mode?	22
2.3.2. Komponentliste - hvad skjules/hvad ændres?	22
3. Databasedesign - implementeringsdetaljer	25
3.1. Forretningsregler implementeret direkte i databaselaget	26
3.1.1. Dagsformens unikhedsbetingelse - @@unique([userId, date])	26
3.1.2. GDPR-sletning via onDelete: Cascade	28
3.1.3. Sammensatte primærnøgler som forretningsregel - @@id([userId, postId])	30
3.1.4. GlobalBan-persistering og e-mail-baseret blokering	31
3.2. Migrerings- og databasehostingsstrategi.....	32
3.2.1. prisma db push vs. prisma migrate - begrundelse for MVP-valget	33
3.2.2. Neon.tech - serverless connection pooling	34
3.2.3. Miljøvariabelhåndtering og Prisma v7-konfiguration.....	35
4. Softwaredesign - implementeringsdetaljer.....	36
4.1. Monorepostruktur og komponentarkitektur	37

4.1.1. Route Handler-arkitektur - tynde controllers og Zod-validering	38
4.1.2. Service-lag og forretningslogik	39
4.1.3. DTO-lag og over-posting-beskyttelse	40
4.2. Autentificering - implementeringsdetaljer	41
4.2.1. Auth.js-konfiguration (src/auth.ts) - overblik	42
4.2.2. signIn-callbacket som primær adgangsvagt	44
4.2.3. JWT- og session-callbacks	46
4.2.4. allowDangerousEmailAccountLinking - bevidst trade-off	47
4.3. Rate limiting og Edge Middleware - implementeringsdetaljer	48
4.3.1. Edge Middleware (src/proxy.ts) - fuldt kodeudsnit	49
4.3.2. Sliding-window-algoritme og differentieret rate limiting	52
4.3.3. Realtids-banvalidering via Redis - setBanFlag og clearBanFlag	54
4.4. Niqo AI-integration - implementeringsdetaljer	55
4.4.1. PII-filer - regex-mønstre og kendte begrænsninger	55
4.4.2. Redis TTL-baseret sessionshåndtering	58
4.4.3. Server-side Gemini API-integration og to-trins-flow	59
4.5. Chat og short-polling - implementeringsdetaljer	62
4.5.1. SWR-baseret polling-konfiguration	62
4.5.2. Samtaleudløb og cron-job (/api/cron/prune-chats)	64
5. Automatiseret deployment - konfigurationsdetaljer	65
5.1. GitLab CI/CD-pipeline	66
5.2. Vercel-deployment - konfiguration og miljøvariabler	69
5.2.1. vercel.json	69
5.2.2. Miljøvariabler i Vercel-projektet	71
5.3. Docker-image og Container Registry	72
5.3.1. Produktionsimage og Docker Hub	72
5.4. Miljøvariabel-strategi - udviklings- vs. produktionsmiljø	73
6. Samlet konklusion	74
7. Referencer	75
8. Bilag	77

Figurliste

Figur 1: Samme landingsside i Light Mode (højre) og Dark Mode (venstre).	6
Figur 2: Komponentdiagram over DailyStatus-systemets tre lag.	11
Figur 3: Skærbilleder der viser interceptordialogvinduet i tre tilstande.	18
Figur 4: Nærbillede af DailyStatusSlider med thumben placeret på trin 3.	21
Figur 5: Side-by-side skærbilleder af feed-visningen i normal tilstand og Low Energy Mode.	23

Tabelliste

Tabel 1: Chroniqos fem dagsforms-farve-tokens med tilhørende CSS custom property-navne.....	4
Tabel 2: Avatarpalettens 16 pastelfarver defineret som CSS custom properties i globals.css.	5
Tabel 3: Chroniqos tre Google Fonts-familier, deres CSS-variabelnavne og Tailwind-tokens.	7
Tabel 4: Navngivne keyframe-animationer eksponeret som Tailwind-utilities.	10
Tabel 5: Konkrete UI-ændringer ved aktivering af Low Energy Mode.....	23
Tabel 6: Sammenligning af prisma db push og prisma migrate dev.	33
Tabel 7: Konfigurationsparametre for de to Upstash-rate limiters i src/lib/upstash.ts.....	53
Tabel 8: SWR polling-intervaller og revalideringsstrategier for chat-hooks.	63
Tabel 9: Oversigt over projektets cron-jobs konfigureret i vercel.json.....	70
Tabel 10: Samtlige miljøvariabler konfigureret i Vercel-projektets produktionsmiljø.	71

Kodeudsnitsliste

Kodeudsnit 1: CSS custom properties for Light Mode og Dark Mode i globals.css.	3
Kodeudsnit 2: @theme inline-blokken i globals.css eksponerer CSS custom properties.	5
Kodeudsnit 3: Next.js font-optimering i src/app/[locale]/layout.tsx.	7
Kodeudsnit 4: themeScript injiceret i <head> via dangerouslySetInnerHTML.	8
Kodeudsnit 5: moodScript injiceret parallelt med themeScript.	8
Kodeudsnit 6: checkStatus-effekten i DailyStatusProvider.	12
Kodeudsnit 7: markRegistered-funktionen i DailyStatusProvider.	14
Kodeudsnit 8: Åbningslogik-effekten i DailyStatusInterceptor.	15
Kodeudsnit 9: Onboarding-prefill-logikken i DailyStatusInterceptor.	16
Kodeudsnit 10: triggerFeedback-funktionen i DailyStatusInterceptor.	17
Kodeudsnit 11: Pointer Events-handlere i DailyStatusSlider.	19
Kodeudsnit 12: computeStep-funktionen i DailyStatusSlider.	20
Kodeudsnit 13: Afledning af isLowEnergy-flag'et fra DailyStatusContext.	22
Kodeudsnit 14: DailyStatus-modellen i prisma/schema.prisma.	26
Kodeudsnit 15: upsertDailyStatus-funktionen i src/services/daily-status.service.ts.	27
Kodeudsnit 16: Post- og Comment-modellerne i prisma/schema.prisma med onDelete:.	28
Kodeudsnit 17: GlobalBan-modellen med onDelete: SetNull.	29
Kodeudsnit 18: PostSupport- og MessageReaction-modellerne i prisma/schema.prisma.	30
Kodeudsnit 19: Den fulde GlobalBan-model i prisma/schema.prisma.	31
Kodeudsnit 20: prisma.config.ts - Prisma v7's adskilte konfigurationsfil.	35
Kodeudsnit 21: Route Handler-mønsteret i src/app/api/posts/route.ts.	38
Kodeudsnit 22: createPost-funktionen i src/services/post.service.ts.	39
Kodeudsnit 23: createPostSchema i src/lib/dtos/post.dto.ts.	40
Kodeudsnit 24: Provider-konfigurationen i src/auth.ts.	42
Kodeudsnit 25: Båndlysningstjekket i signIn-callbacket.	44
Kodeudsnit 26: OAuth signup-verifikationslogikken.	45
Kodeudsnit 27: jwt-callbacket i src/auth.ts.	46
Kodeudsnit 28: Den fulde proxy-funktion i src/proxy.ts.	50
Kodeudsnit 29: rateLimiter og rateLimiterPerHour i src/lib/upstash.ts.	52
Kodeudsnit 30: setBanFlag og clearBanFlag i src/lib/upstash.ts.	54

Kodeudsnit 31: filterPII-funktionen i src/lib/utils/pii-filter.ts med de fem filterlag.	56
Kodeudsnit 32: SYSTEM_PROMPT-konstanten i src/services/niqo.service.ts.	57
Kodeudsnit 33: startNewChat-funktionen i src/services/niqo.service.ts.	58
Kodeudsnit 34: generateResponse-funktionen i src/services/niqo.service.ts.....	60
Kodeudsnit 35: De fire primære polling-hooks i src/lib/hooks/use-chat.ts.	62
Kodeudsnit 36: Cron-logikken i /api/cron/prune-chats.....	64
Kodeudsnit 37: Den komplette .gitlab-ci.yml.....	68
Kodeudsnit 38: Den komplette vercel.json i projektets rod.....	69

1. Indledning og formål

Dette dokument uddyber implementeringsdetaljerne bag afsnit 6 (Design og Implementering) i hovedrapporten. Formålet er at give adgang til de tekniske dybder - kodeudsnit, konfigurationsfiler og detaljerede implementeringsbeskrivelser - uden at disse belaster rapportens røde tråd. Rapporten beskriver *hvorfor* de pågældende designbeslutninger er truffet; dette bilag beskriver *hvordan* de er implementeret i kodebasen.

Bilaget er struktureret parallelt med afsnit 6 og udvider hvert underafsnit med det tekniske indhold, der er relevant, hvis der ønskes at inspicere, reproducere eller videreudvikle systemet. Ønskes den fulde kildekode inspiceret eller afviklet lokalt, fremgår den af Bilag 9 (Kildekode) (1).

Bilag 6 bør læses i sammenhæng med de øvrige designbilag: Bilag 5 (Systemdesign og Arkitektur) (2) dokumenterer hvad systemet er designet til, herunder arkitekturdiagrammer og databasemodellen. Bilag 7 (Test & Validering) (3) dokumenterer, at implementeringen er korrekt. Bilag 8 (UI-Dokumentation) (4) viser, hvordan systemet ser ud. Bilag 6 - dette bilag - dokumenterer hvordan systemet er bygget: kodeimplementeringer, konfigurationsfiler og tekniske valg.

2. UI/UX-design - implementeringsdetaljer

Chroniqos brugergrænseflade er bygget oven på et centralt designsystem, der sikrer visuel konsistens på tværs af alle komponenter. Dette afsnit dokumenterer implementeringsdetaljerne bag designsystemets tre nøglekomponenter: CSS-variabel-arkitekturen i `globals.css`, dagsformssystemets komponentlag og det adaptive UI, der automatisk tilpasser sig brugerens registrerede energiniveau.

2.1. Designsystem og CSS-variabler (globals.css)

Chroniqos designsystem er implementeret udelukkende i `src/app/globals.css` og anvender Tailwind CSS v4's `@theme inline`-mekanisme (5). Frem for den tidligere `tailwind.config.ts` defineres alle designbeslutninger som CSS custom properties direkte i stylesheet-filen, der derefter eksponeres som Tailwind-utilities via `@theme inline`-blokken. Resultatet er, at komponenter kan anvende klassenavne som `bg-brand`, `text-foreground` og `bg-surface-opaque` uden at vide, hvilken konkret farveværdi der ligger bag - tokenet løser det.

2.1.1. Farvepalet og temavariabler (Light-/Dark Mode)

Temaet implementeres ved hjælp af to CSS-regelsæt: `:root` definerer *Light Mode*-tokens, og `.dark`-selectoren overskriver de tokens der ændrer sig i mørkt tema. Tokens der er identiske på tværs af temaer - primært dagsformsfarver og avatarpaletten - defineres kun i `:root`.

```
1. // src/app/globals.css
2. /* Light Mode Tokens */
3. :root {
4.   --background:      #f7f7f5;
5.   --foreground:      #121212;
6.   --foreground-muted: #12121277;
7.   --brand:           #e65c69; /* Vibrant Care - primær CTA-farve */
8.   --brand-muted:     #f0939c;
9.
10.  --surface:          #eede9;
11.  --surface-border:   #00000012;
12.  --surface-opaque:   #eede9;
13.  --surface-hover:    #e3e2de;
14.
15.  --navbar-bg:        #f7f7f5e6; /* Gennemsigtigt for backdrop-blur-effekt */
16.  --feedback-success: #9baf64;
17.  --feedback-error:   #e65c69;
18. }
19.
20. /* Dark Mode Tokens - kun overrides */
21. .dark {
22.   --background:      #121212;
23.   --foreground:      #f7f7f5;
24.   --surface:         #ffffff0d; /* Semitransparent white for glassmorphism */
25.   --surface-opaque:  #1e1e1e;
26.   --surface-hover:   #2a2a2a;
27.   --navbar-bg:       #121212e6;
28. }
```

Kodeudsnit 1: CSS custom properties for Light Mode (`:root`) og Dark Mode (`.dark`) i `globals.css`. Dark Mode definerer kun de tokens der ændrer sig - øvrige tokens arves uændret fra `:root`.

Dagsformsfarver er temaagnostiske og defineres identisk i begge temaer:

Tabel 1: Chroniqos fem dagsforms-farve-tokens med tilhørende CSS custom property-navne, hex-værdier, dagsforms-værdi (nulindeksret) og dansk betegnelse som vist i brugergrænsefladen.

Dagsforms-farve-tokens - CSS custom properties og tilhørende dagsformsværdier			
Token	Hex	Dagsform	Dansk betegnelse
--color-dailystatus-exhausted	#A591F5	1 (0 i kode)	Helt Udmattet
--color-dailystatus-low-energy	#EB7D19	2 (1 i kode)	Lavt Overskud
--color-dailystatus-neutral	#915F46	3 (2 i kode)	Nogenlunde
--color-dailystatus-good-periods	#FFCD5A	4 (3 i kode)	Gode Perioder
--color-dailystatus-full-energy	#9BAF64	5 (4 i kode)	Fuldt Overskud

Bemærk om indeksering: Dagsformsværdier gemmes i databasen som 0-4 (nulindeksret), mens de præsenteres for brugeren som 1-5. Alle tokens og komponenter opererer konsekvent med den nulindekserede repræsentation.

Avatarpaletten er 16 pastelfarver der bruges til brugergenererede avatar- og fællesskabsikoner. Farverne gemmes som hex-værdier i databasen og er designet til at fungere på begge temaer uden ændringer:

Tabel 2: Avatarpalettens 16 pastelfarver defineret som CSS custom properties i `globals.css`. Farverne gemmes som hex-strengene i databasen og er designet til at fungere på begge temaer uden ændringer.

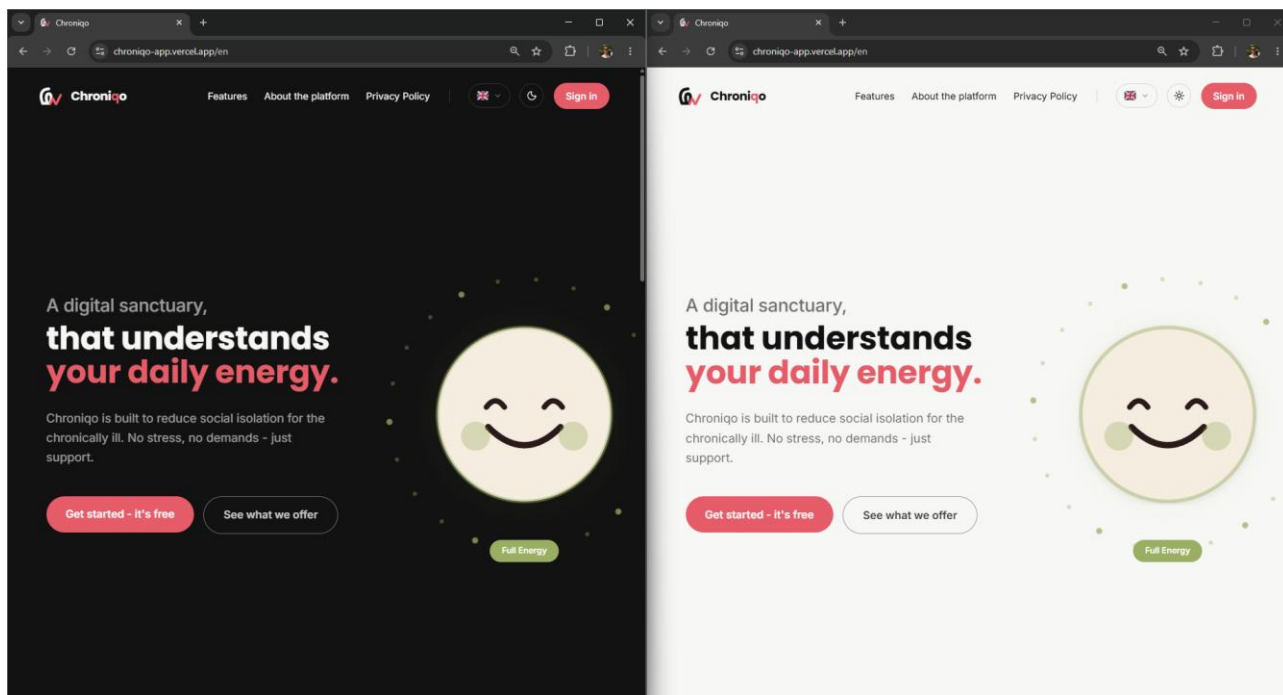
Avatarpalettens 16 pastelfarver - token-navne og hex-værdier			
Token	Hex	Token	Hex
<code>--palette-rose</code>	<code>#f4b8c1</code>	<code>--palette-sky</code>	<code>#a8d4f5</code>
<code>--palette-coral</code>	<code>#f7c5a8</code>	<code>--palette-periwinkle</code>	<code>#b8b8f4</code>
<code>--palette-peach</code>	<code>#fad9b0</code>	<code>--palette-lavender</code>	<code>#d1b8f4</code>
<code>--palette-butter</code>	<code>#f9e8a0</code>	<code>--palette-lilac</code>	<code>#f4b8e8</code>
<code>--palette-sage</code>	<code>#bfdc9e</code>	<code>--palette-smoke</code>	<code>#d4d0cc</code>
<code>--palette-mint</code>	<code>#a8ddc8</code>	<code>--palette-slate</code>	<code>#b0bcc8</code>
<code>--palette-sand</code>	<code>#e8dcc8</code>	<code>--palette-charcoal</code>	<code>#3a3a3a</code>
<code>--palette-stone</code>	<code>#c8c0b4</code>	<code>--palette-midnight</code>	<code>#1e2a3a</code>

`@theme inline`-blokken eksponerer samtlige tokens som Tailwind-utilities:

```
1. // src/app/globals.css
2. @theme inline {
3.   --color-background:    var(--background);
4.   --color-brand:         var(--brand);
5.   --color-surface:       var(--surface);
6.   --color-dailystatus-exhausted: #a591f5;
7.   /* ... samtlige tokens kortlægges her */
8.
9.   --font-sans:    var(--font-inter);
10.  --font-heading: var(--font-poppins);
11. }
```

Kodeudsnit 2: `@theme inline`-blokken i `globals.css` eksponerer CSS custom properties som Tailwind v4-utilities. Kortlægningen muliggør klasser som `bg-brand` og `text-foreground` der automatisk løser til det aktuelle temas farveværdi.

Resultatet er, at et HTML-element som `<div className="bg-surface text-foreground border-surface-border">` automatisk skifter udseende ved temaskift uden ændringer i selve komponentkoden.



Figur 1: Samme landingsside i Light Mode (højre) og Dark Mode (venstre). Baggrunden, tekst og kantlinjer skifter automatisk ved temaskift, da alle farveværdier er defineret som CSS custom properties (`--background`, `--surface`, `--foreground`, `--surface-border`) der overstyres af `.dark`-sektoren i `globals.css`. Komponentkoden er uændret mellem de to temaer.

2.1.2. Typografisk skala (Poppins, Inter og Syne)

Tre Google Fonts-familier indlæses i `src/app/[locale]/layout.tsx` og registreres som CSS-variabler via Next.js' font-optimering:

```
1. // src/app/[locale]/layout.tsx
2. const syne = Syne({
3.   variable: "--font-syne",
4.   subsets: ["latin"],
5.   weight: ["800"],
6. });
7.
8. const poppins = Poppins({
9.   variable: "--font-poppins",
10.  subsets: ["latin"],
11.  weight: ["600", "700"],
12. });
13.
14. const inter = Inter({
15.   variable: "--font-inter",
16.   subsets: ["latin"],
17.   weight: ["400", "500", "600", "700"],
18. });
```

Kodeudsnit 3: Next.js font-optimering i `src/app/[locale]/Layout.tsx`. De tre Google Fonts-familier indlæses med `next/font/google`, der genererer en unik CSS-variabel per font og automatisk self-hoster skrifttypen som en statisk asset - ingen eksternt fontnetværkskald ved render (6).

De tre skrifttyper har distinkte roller i designsystemet:

Tabel 3: Chroniqos tre Google Fonts-familier, deres CSS-variabelnavne, tilhørende Tailwind-tokens, indlæste vægte og anvendelsesområde i designsystemet.

Chroniqos tre Google Fonts-familier				
Font	CSS-variabel	Tailwind-token	Vægte	Anvendelse
Inter	--font-inter	font-sans	400 - 700	Brødtekst, UI-elementer, input-felter, knapper
Poppins	--font-poppins	font-heading	600 - 700	Overskrifter (font-heading), dialogvindue-titler, fremhævet UI
Syne	--font-syne	font-syne	800	Udelukkende landingssiden: den store hero-headline

Syne er valgt til hero-headlinen for dens markante geometriske karakter og ultra-tykke 800-vægt, der adskiller landingssiden visuelt fra appens interne UI, som konsekvent bruger Poppins og Inter. `font-sans` og `font-heading` kortlægges i `@theme inline`-blokken, mens `--font-syne` bruges direkte med Tailwind-klassen `font-syne` der matcher variabelnavnet.

2.1.3. Temainitialisering (forebyggelse af FOUC)

Et Flash of Unstyled Content (FOUC) opstår, når React hydrerer med et tema der adskiller sig fra det der er gemt i brugerens `localStorage` (7). For at forebygge dette injiceres to inline `<script>`-blokke i `<head>` i `layout.tsx`, der eksekveres synkront *før* første render:

`themeScript` - læser `localStorage['theme-preference']` og tilføjer `.dark`-klassen til `<html>`-elementet hvis brugerens præference eller OS-indstilling kræver det:

```
1. // src/app/[locale]/layout.tsx
2. const themeScript = `
3.   (function() {
4.     try {
5.       var storedTheme = localStorage.getItem('theme-preference');
6.       var prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
7.       if (storedTheme === 'dark' ||
8.         (storedTheme === 'system' && prefersDark) ||
9.         (!storedTheme && prefersDark)) {
10.        document.documentElement.classList.add('dark');
11.      } else {
12.        document.documentElement.classList.remove('dark');
13.      }
14.    } catch (e) {}
15.  })();
16. `;
```

Kodeudsnit 4: `themeScript` injiceret i `<head>` via `dangerouslySetInnerHTML`. Scriptet eksekveres synkront før første render og tilføjer `.dark`-klassen til `<html>`-elementet baseret på `localStorage['theme-preference']` og OS-præferencen (`prefers-color-scheme`).

`moodScript` - læser `localStorage['chroniqo-mood']` og sætter `--brand`-variablen på `<html>`-elementet til brugerens dagsformsfarve, hvis registreringen er fra dags dato:

```
1. // src/app/[locale]/layout.tsx
2. const moodScript = `
3.   (function() {
4.     try {
5.       var stored = JSON.parse(localStorage.getItem('chroniqo-mood') || 'null');
6.       var today = new Date().toLocaleDateString('en-CA');
7.       var colors = ['#A591F5', '#EB7D19', '#915F46', '#FFCD5A', '#9BAF64'];
8.       if (stored && stored.date === today &&
9.         stored.value >= 0 && stored.value <= 4) {
10.        document.documentElement.style.setProperty('--brand', colors[stored.value]);
11.      }
12.    } catch (e) {}
13.  })();
14. `;
```

Kodeudsnit 5: `moodScript` injiceret parallelt med `themeScript`. Scriptet validerer at den gemte dagsformsregistrering er fra dags dato (formateret som `YYYY-MM-DD` via `en-CA`-locale) og sætter `--brand`-variablen til den tilhørende dagsformsfarve inden første render.

`en-CA`-locale bruges til `toLocaleDateString` fordi det returnerer formatet `YYYY-MM-DD` konsistent på tværs af browsere og operativsystemer - en detalje der ellers kan forårsage datofortolkningsfejl i Safari og iOS.

`suppressHydrationWarning` på `<html>`-elementet er nødvendigt, fordi `themeScript` muterer `className`-attributten direkte i DOM'en inden React hydrerer. Uden denne attribut vil React kaste en hydration mismatch-advarsel i konsollen, selv om adfærden er korrekt.

2.1.4. Animationsbibliotek

`globals.css` definerer et sæt navngivne keyframe-animationer eksponeret som Tailwind-utilities via `@theme inline`. Disse bruges konsistent på tværs af alle komponenter frem for ad hoc inline-styles:

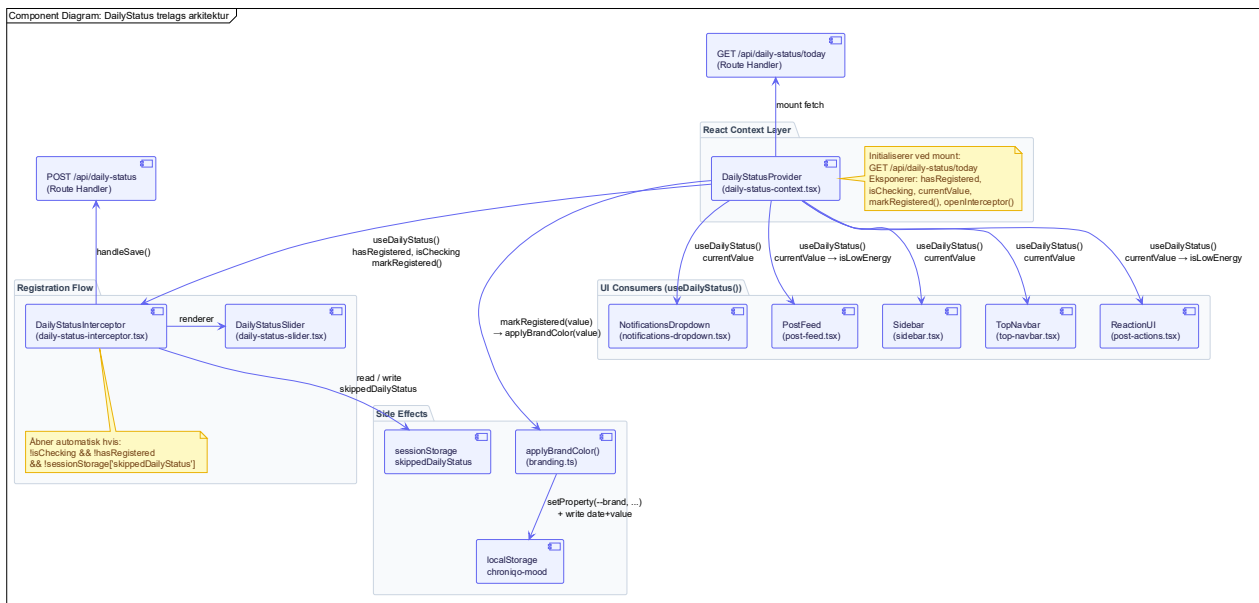
Tabel 4: Navngivne keyframe-animationer eksponeret som Tailwind-utilities via `@theme inline` i `globals.css` - varighed, easing og anvendelsesområde for hver animation.

Navngivne keyframe-animationer eksponeret som Tailwind-utilities			
Utility	Varighed	Easing	Anvendelse
<code>animate-dialog-in / animate-dialog-out</code>	200ms	<code>cubic-bezier(0.16, 1, 0.3, 1)</code>	Alle Radix Dialog-vinduer
<code>animate-dropdown-in / animate-dropdown-out</code>	150ms / 100ms	Spring / ease-in	Dropdown-menuer
<code>animate-reaction-menu-in / out</code>	200ms / 150ms	Spring / ease-in	Emoji-reaktionsmenu
<code>animate-emoji-picker-in / out</code>	200ms / 150ms	Spring / ease-in	Emoji-picker
<code>animate-post-enter</code>	480ms	<code>cubic-bezier(0.16, 1, 0.3, 1)</code>	Indlæsning af opslag i feed
<code>animate-smiley-entrance</code>	800ms	Spring	Smiley-komponentens entrée
<code>animate-gradient-drip</code>	600ms	Spring	Niqo-sidens sideovergange
<code>animate-spin-slow</code>	18s	linear infinite	Roterende cirkler bag Smiley-hero

`cubic-bezier(0.16, 1, 0.3, 1)` er den primære spring-easing der går igen i alle interaktive animationer. Værdien er identisk med Radix UIs interne standardeasing og sikrer visuel konsistens mellem brugerdefinerede og biblioteksbaserede animationer.

2.2. Dagsformsindikatoren - komponentimplementering

Dagsformssystemet er implementeret som tre lag, der samarbejder: en React Context (**DailyStatusProvider**) der holder den globale tilstand, et dialogvindue-komponent (**DailyStatusInterceptor**) der håndterer registreringsflowet, og en slider-komponent (**DailyStatusSlider**) der leverer selve brugergrænsefladen til valg af dagsformsværdi:



Figur 2: Komponentdiagram over **DailyStatus**-systemets tre lag. **DailyStatusProvider** omslutter hele det beskyttede komponenttræ og distribuerer tilstand via `useDailyStatus()`-hook'en til alle forbrugere. **DailyStatusInterceptor** og **DailyStatusSlider** udgør registreringsflowet; UI-konsumenterne (**NotificationsDropdown**, **PostFeed**, **Sidebar**, **TopNavbar**, **ReactionUI**) anvender `currentValue` til at aktivere Low Energy Mode lokalt. `applyBrandColor()` opdaterer `brand-CSS`-variablen og `localStorage` som sideeffekt ved registrering.

2.2.1. DailyStatusContext - tilstandsstyring og distribution

DailyStatusProvider (i `src/context/daily-status-context.tsx`) er applikationens centrale tilstandsholder for dagsformsdata og indkapsler hele det beskyttede komponenttræ. Ved mount foretager den et enkelt API-kald til `/api/daily-status/today` for at afgøre om brugeren allerede har registreret sin dagsform den gældende dag:

```
1. // src/context/daily-status-context.tsx
2. useEffect(() => {
3.   const checkStatus = async () => {
4.     try {
5.       const res = await fetch("/api/daily-status/today");
6.       if (res.ok) {
7.         const data = await res.json();
8.         setHasRegistered(data.hasRegistered);
9.
10.        const value: number | undefined = data.status?.value;
11.        if (typeof value === "number") {
12.          setCurrentValue(value);
13.          applyBrandColor(value); // Opdaterer --brand CSS-variablen øjeblikkeligt
14.        } else {
15.          applyBrandColor(null); // Nulstiller til standard #e65c69
16.        }
17.      }
18.    } catch (err) {
19.      console.error("Failed to check daily status", err);
20.    } finally {
21.      setIsChecking(false);
22.    }
23.  };
24.  checkStatus();
25. }, []);
```

Kodeudsnit 6: checkStatus-effekten i DailyStatusProvider (src/context/daily-status-context.tsx). Ved mount foretages ét API-kald til `/api/daily-status/today`; resultatet afgør `hasRegistered`-tilstanden og opdaterer `--brand`-variablen synkront via `applyBrandColor`. `catch`-blokken logger fejlen uden at blokere applikationen - `isChecking` sættes altid til `false` i `finally`-blokken uanset udfald.

`isChecking` sættes til `true` ved initialisering og til `false` i `finally`-blokken. Denne tilstand bruges af `DailyStatusInterceptor` til at undgå at åbne registreringsdialogvinduet, inden API-svaret er returneret - uden dette tjek ville dialogvinduet åbne prematurt for brugere der allerede har registreret.

`applyBrandColor(value)` er en hjælpefunktion i `src/lib/utils/branding.ts` der kalder `document.documentElement.style.setProperty('--brand', colors[value])` og opdaterer `localStorage['chroniqo-mood']` med dagsformsværdi og dags dato. Derved synkroniseres den persisterede farve med `moodScript`-logikken i `layout.tsx` og eliminerer FOUC ved næste sideindlæsning.

`markRegistered(value)` kaldes fra `DailyStatusInterceptor` umiddelbart efter en succesfuld API-respons. Funktionen opdaterer `hasRegistered`, `currentValue` og kalder `applyBrandColor` synkront, hvilket betyder at brandfarven skifter øjeblikkeligt i UI'et uden at afvente en ny API-fetch. Derudover invaliderer den alle SWR-cacheindgange der matcher `/api/conversations`, da samtalevisningen viser brugerens aktuelle dagsformsstatus i sin header:

```
1. // src/context/daily-status-context.tsx
2. const markRegistered = (value?: number) => {
3.   setHasRegistered(true);
4.   if (typeof value === "number") {
5.     setCurrentValue(value);
6.     applyBrandColor(value);
7.   }
8.   globalMutate(
9.     (key: unknown) =>
10.      typeof key === "string" && key.includes("/api/conversations"),
11.   );
12. };
```

Kodeudsnit 7: markRegistered-funktionen i DailyStatusProvider. Opdaterer hasRegistered, currentValue og brandfarven synkront efter en succesfuld registrering, og invaliderer SWR-cacheindgange for /api/conversations da samtale-headeren viser brugerens aktuelle dagsformsstatus.

Konteksten eksponerer desuden tre debug-hjælpere (debugForceOpen, debugTestSuccess, debugTestFailure) der bruges af systemadministratorer til at afprøve interceptor-flowet og dens animations-tilstande uden at slette en reel dagsformsregistrering i databasen.

2.2.2. DailyStatusInterceptor - dialogvindue og registreringsflow

DailyStatusInterceptor (i `src/app/[locale]/(protected)/(components)/daily-status-interceptor.tsx`) er det dialogvindue der automatisk præsenteres ved login, når brugeren endnu ikke har registreret sin dagsform på den gældende dag. Den er implementeret som en Radix Dialog, der renderes i en React Portal og dermed er immun over for eventuelle overflow: hidden-begrænsninger i forældrekomponenterne (8).

2.2.2.1. Åbningslogik

Interceptoren åbner sig selv baseret på tre betingelser der alle skal være opfyldt:

```
1. // src/app/[locale]/(protected)/(components)/daily-status-interceptor.tsx
2. useEffect(() => {
3.   if (!isChecked && !hasRegistered) {
4.     const hasSkipped = sessionStorage.getItem("skippedDailyStatus");
5.     if (hasSkipped !== "true") {
6.       // Guard: åbn ikke hvis RememberMeWatcher er ved at afmelde brugeren
7.       const hasSession = sessionStorage.getItem("chroniqo_session");
8.       if (!hasSession) {
9.         const rememberData = localStorage.getItem("chroniqo_remember");
10.        const expiry = rememberData ? parseInt(rememberData, 10) : NaN;
11.        if (isNaN(expiry) || Date.now() > expiry) {
12.          return; // RememberMeWatcher håndterer redirect
13.        }
14.      }
15.      setIsOpen(true);
16.    }
17.  }
18. }, [isChecked, hasRegistered, resetTrigger]);
```

Kodeudsnit 8: Åbningslogik-effekten i DailyStatusInterceptor. Dialogvinduet åbner når `isChecked` er `false`, `hasRegistered` er `false` og `sessionStorage['skippedDailyStatus']` ikke er sat. Guarden i bunden forhindrer dialogvinduet i at åbne hvis `RememberMeWatcher` er ved at afmelde brugeren.

Betingelserne er:

1. API-tjekket er afsluttet (`!isChecked`).
2. Brugeren har ikke registreret endnu (`!hasRegistered`).
3. Brugeren har ikke sprunget over i denne session (`sessionStorage['skippedDailyStatus'] !== 'true'`).

Derudover er der en guard der forhindrer dialogvinduet i at åbne, hvis `RememberMeWatcher`-komponenten er ved at afmelde brugeren - Dialog-portaler renderes til `document.body` og ville ellers vises oven på afmeldingsflowet.

2.2.2.2. Forudfyldning fra onboarding

Når dialogvinduet åbner, hentes dagens registrering fra `/api/daily-status/today`. Hvis brugeren ikke har registreret endnu, tjekkes `localStorage['chroniqo_onboarding_mood']` - et felt der sættes under onboardingflowet, hvor brugeren allerede har angivet sit energiniveau. Er nøglen til stede, præudfyldes slider og notefelt med onboarding-værdien, og nøglen slettes fra `localStorage` for at forhindre prefill gengives unødigt ved fremtidige åbninger:

```
1. // src/app/[locale]/(protected)/(components)/daily-status-interceptor.tsx
2. const onboardingPrefill = localStorage.getItem("chroniqo_onboarding_mood");
3. if (onboardingPrefill) {
4.   try {
5.     const parsed = JSON.parse(onboardingPrefill);
6.     setStatusValue(parsed.value ?? 2);
7.     setNote(parsed.note ?? "");
8.   } catch {
9.     if (interceptorInitialValue === null) setStatusValue(2);
10.    setNote("");
11.  }
12.  localStorage.removeItem("chroniqo_onboarding_mood");
13. }
```

Kodeudsnit 9: Onboarding-prefill-logikken i `DailyStatusInterceptor`. Hvis `localStorage['chroniqo_onboarding_mood']` er til stede og endnu ikke brugt, præudfyldes slider og note-felt med den stemning brugeren angav under onboarding. `JSON.parse`-kaldet er indkapslet i `try/catch` for at beskytte mod korrupt `localStorage`-data - `catch`-blokken nulstiller til standardværdier. Nøglen slettes efter `try/catch`-blokken for at forhindre gen-prefill uanset om parsing lykkedes.

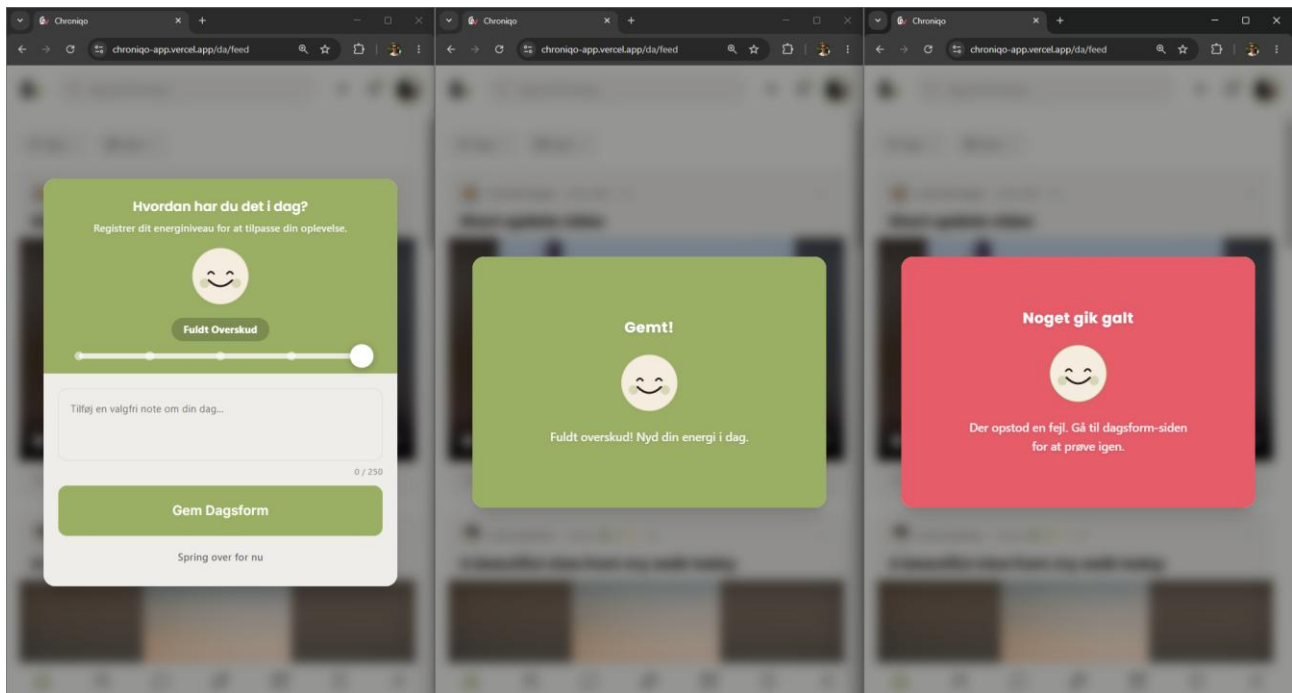
2.2.2.3. Gem-flow og animeret feedback

Ved gem sendes en POST-request til `/api/daily-status` med den valgte værdi og eventuelt en note. Afhængigt af API-responsens statuskode kalder komponenten `triggerFeedback('success')` eller `triggerFeedback('error')`, der eksekverer en to-trins-animation:

```
1. // src/app/[locale]/(protected)/(components)/daily-status-interceptor.tsx
2. const triggerFeedback = (state: "success" | "error") => {
3.   const currentH = formRef.current?.scrollHeight ?? 420;
4.   setContainerHeight(currentH); // Lås højden på formularens aktuelle højde
5.
6.   setSaveState(state);
7.
8.   requestAnimationFrame(() => {
9.     requestAnimationFrame(() => {
10.      setContainerHeight(FEEDBACK_HEIGHT); // Animer til 340 px
11.      setShowingFeedback(true);
12.      setTimeout(() => setFeedbackVisible(true), 50);
13.    });
14.  });
15.
16.  const delay = state === "success" ? 4000 : 3200;
17.  setTimeout(() => {
18.    if (state === "success") markRegistered(statusValue);
19.    setIsOpen(false);
20.    setTimeout(resetState, 300);
21.  }, delay);
22. };
```

Kodeudsnit 10: triggerFeedback-funktionen i DailyStatusInterceptor. Det dobbelte requestAnimationFrame-kald tvinger browseren til at renderere den låste starthøjde i én frame, inden CSS-overgangen til FEEDBACK_HEIGHT (340 px) igangsættes i den næste. Uden dette ville overgangen ikke have et startpunkt at animere fra.

Det dobbelte `requestAnimationFrame`-kald er nødvendigt for at tvinge browseren til at renderere den låste højde i én frame, inden den animerede højdeændring igangsættes i den næste. Uden dette ville CSS-overgangen ikke have noget startpunkt at animere fra, og højdeanimationen ville ikke udløses. `FEEDBACK_HEIGHT` er fastsat til 340 px - en konstant der er kalibreret til at smiley-ikonet og feedback-teksten passer komfortabelt uden scroll på alle skærmstørrelser.



Figur 3: Skærbilleder der viser interceptor-dialogvinduet tre tilstande side om side: (1) Formular-tilstand med slider, noter og gem/spring-knapper. (2) Success-feedback-overlay med Smiley og positiv besked, farvet med brugerens valgte dagsformsfarve. (3) Error-feedback-overlay med rød feedback-farve.

2.2.3. DailyStatusSlider - brugergrænsefladekomponent

DailyStatusSlider (i `src/app/(components)/daily-status-slider.tsx`) er en brugerdefineret slider-komponent der erstatter det native `<input type="range">`. Valget af en brugerdefineret implementering frem for den native HTML-slider skyldes, at `<input type="range">` har begrænsede stylingsmuligheder på tværs af browsere, bl.a. for thumb- og track-elementet i Safari og Firefox.

Komponenten bruger Pointer Events API i stedet for de traditionelle mouse/touch-events (9):

```
1. // src/app/(components)/daily-status-slider.tsx
2. const onPointerDown = (e: React.PointerEvent<HTMLDivElement>) => {
3.   dragging.current = true;
4.   e.currentTarget.setPointerCapture(e.pointerId);
5.   computeStep(e.clientX);
6. };
7.
8. const onPointerMove = (e: React.PointerEvent<HTMLDivElement>) => {
9.   if (dragging.current) computeStep(e.clientX);
10. };
```

Kodeudsnit 11: Pointer Events-handlere i DailyStatusSlider. `setPointerCapture` fastholder, at elementet modtager alle efterfølgende pointer-events selv om markøren bevæger sig uden for track-elementets bounding box - kritisk for korrekt drag-adfærd på touchskærme og ved hurtige musebevægelser.

`setPointerCapture(e.pointerId)` er centralt: det fastholder, at det aktuelle element modtager alle efterfølgende pointer-events selv om markøren bevæger sig uden for track-elementets bounding box. Uden dette ville en hurtig drag-bevægelse resultere i, at `onPointerMove` ikke udløses, og thumben slipper midtvejs. En `ref` (`dragging`) bruges frem for `state` til at spore drag-tilstanden, da `state`-opdateringer er asynkrone og ville introducere race conditions under hurtige drag-bevægelser.

`computeStep` beregner det nærmeste trin baseret på markørens horisontale position relativt til track-elementets bounding box:

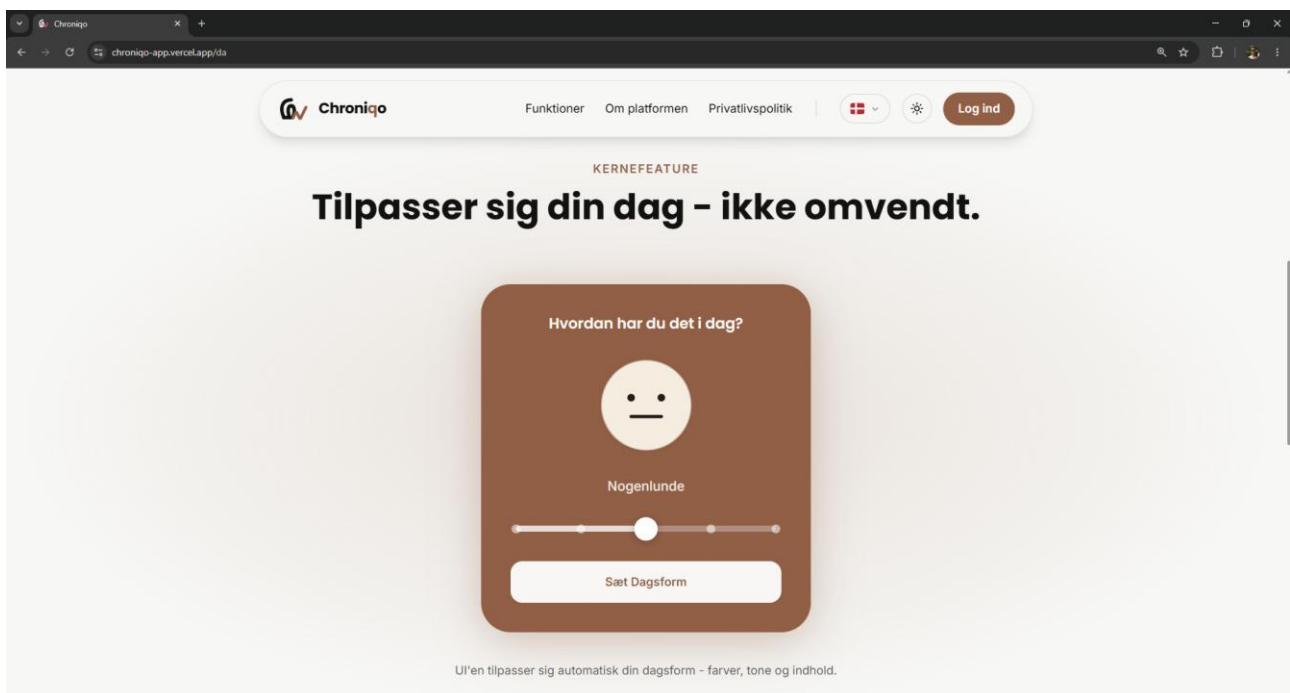
```
1. // src/app/(components)/daily-status-slider.tsx
2. const computeStep = useCallback(
3.   (clientX: number) => {
4.     if (!trackRef.current) return;
5.     const rect = trackRef.current.getBoundingClientRect();
6.     const pct = Math.max(0, Math.min(1, (clientX - rect.left) / rect.width));
7.     onChange(Math.round(pct * 4));
8.   },
9.   [onChange],
10. );
```

Kodeudsnit 12: `computeStep`-funktionen i `DailyStatusSlider`. Konverterer markørens absolutte X-koordinat til et af de fem trin (0-4) via procentberegning mod trackens bounding box. `Math.max/min`-afgrænsningen forhindrer out-of-bounds-indeksering ved drag ud over kanten af track-elementet.

`Math.round(pct * 4)` kvantiserer den kontinuerte procent til et af de fem trin (0-4). `Math.max(0, Math.min(1, ...))` begrænser værdien til intervallet [0, 1] og forhindrer out-of-bounds-indeksering selv ved drag ud over kanten.

Visuelt består slider af tre lag (alle positioneret `absolute` inden for en `relative` container):

1. **Fill-bar** (`bg-white/70`): bredde beregnet som $(\text{value} / 4) * 100\%$ med CSS `transition-all duration-300`.
2. **Step-dots**: fem `<div>`-elementer positioneret ved `left: 0%, 25%, 50%, 75%, 100%`. Det aktive trin anvender `.active`-klassen fra `globals.css`, der forstørrer dot'en til 18x18 px og tilføjer en hvid glow-ring.
3. **Thumb** (`bg-white, w-8 h-8, rounded-full`): positioneret identisk med fill-bar og animeres med `transition-all duration-300`.



Figur 4: Nærbillede af `DailyStatusSlider` med thumben placeret på trin 3 (Nogenlunde). Figuren viser step-dots, fill-bar og thumb tydeligt. Slider-komponentens farvetemperatur matcher den aktive dagsformsfarve i containerens baggrund.

2.3. Det adaptive UI (Low Energy Mode) - implementeringsdetaljer

Low Energy Mode er implementeret som en rent client-side tilstandsmaskine uden serversidelogik: afsnit 2.3.1 beskriver præcis under hvilke betingelser tilstanden aktiveres ud fra `DailyStatusContext`, mens afsnit 2.3.2 dokumenterer de konkrete UI-ændringer der træder i kraft - fra undertrykkelse af notifikationsbadges til erstatning af infinite scroll med paginering.

2.3.1. Tilstandstransition - hvornår aktiveres Low Energy Mode?

Low Energy Mode aktiveres, når `currentValue` fra `DailyStatusContext` er `0` eller `1`, svarende til dagsformsniveauerne Helt Udmattet og Lavt Overskud. Komponenter på tværs af applikationen henter tilstanden via `useDailyStatus()`-hook'en:

```
1. const { currentValue } = useDailyStatus();  
2. const isLowEnergy = currentValue !== null && currentValue <= 1;
```

Kodeudsnit 13: Afledning af `isLowEnergy`-flag'et fra `DailyStatusContext`. `currentValue !== null`-tjekket sikrer, at fraværet af en dagsformsregistrering ikke fortolkes som lav energi - Low Energy Mode aktiveres udelukkende ved en eksplicit registrering på niveau `0` eller `1`.

`currentValue !== null`-tjekket er afgørende: `null` indikerer, at API-tjekket endnu ikke er returneret, eller at brugeren ikke har registreret dagsform i dag. I begge tilfælde renderes den fulde UI-oplevelse, da fraværet af en registrering ikke er ensbetydende med lav energi. *Low Energy Mode* aktiveres udelukkende ved en eksplicit registrering på niveau `0` eller `1`.

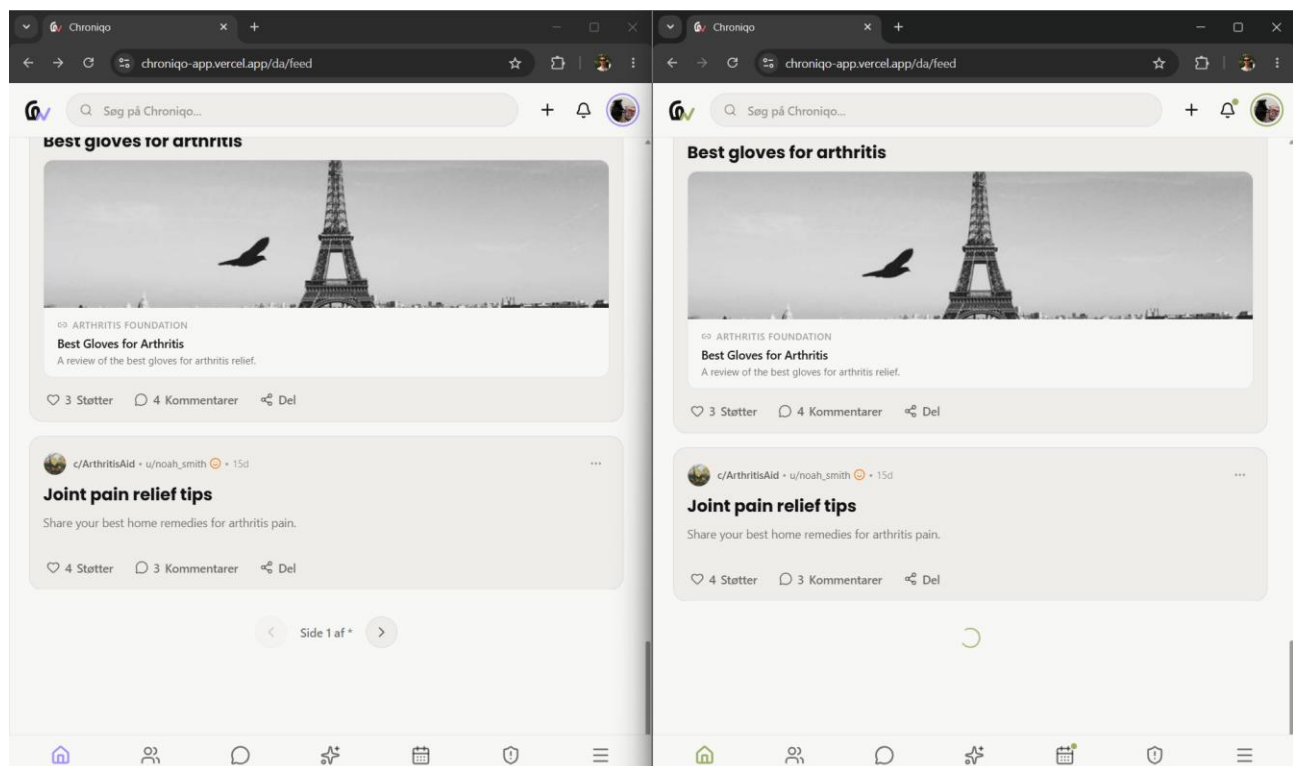
Tilstanden er rent client-side og kræver ingen ekstra API-kald - `currentValue` er allerede tilgængeligt i Context fra den initiale `checkStatus`-fetch ved login.

2.3.2. Komponentliste - hvad skjules/hvad ændres?

Tabel 5 dokumenterer de konkrete UI-ændringer *Low Energy Mode* medfører. Samtlige ændringer er implementeret som betingede renders eller betingede Tailwind-klasser baseret på `isLowEnergy`-flaget, og ingen kræver separat API-kald eller serversidelogik.

Tabel 5: Konkrete UI-ændringer ved aktivering af Low Energy Mode ($currentValue \leq 1$) - komponent, normal tilstand, Low Energy-tilstand og den implementeringsmetode der aktiverer ændringen.

Konkrete UI-ændringer ved aktivering af Low Energy Mode			
Komponent	Normal tilstand	Low Energy Mode	Implementering
notifications-drop-down.tsx	Rød badge-tæller vises på notifikations-ikonet	Badge skjules - ikonet vises uden tæller	<code>!isLowEnergy</code> check på badge-rendering
post-feed.tsx	Infinite scroll via <code>react-virtuoso</code>	Pagineret feed med forrige/næste knapper	<code>if (isLowEnergy)</code> slicing af posts + pagination UI
sidebar.tsx	Unread message badge vises på messages ikon	Badge skjules	<code>!isLowEnergy</code> && før badge-rendering
floating-chat.tsx	Rød badge-tæller vises på chat-ikonet	Badge skjules	<code>!isLowEnergy</code> && før badge-rendering



Figur 5: Side-by-side skærbilleder af feed-visningen i Normal Mode (højre) og Low Energy Mode (venstre). Normal Mode viser infinite scroll-indikator og fuld notifikationsbadges. Low Energy Mode viser "Indlæs flere"-knap og badge uden tæller.

Den paginerede feed-implementering i *Low Energy Mode* anvender samme `post-feed`-komponent men med en eksplicit `page-state` frem for `react-virtuoso`'s virtuelle scroll-container (10). Dette reducerer antallet af samtidige DOM-noder markant, da virtuel scroll-rendering kræver løbende rekalkulering af elementpositioner ved scroll - en CPU-intensiv operation der er u hensigtsmæssig for brugere med lav energi og potentielt reduceret kognitiv kapacitet.

Valget af betingede renders frem for CSS-visibility (`visibility: hidden / display: none`) er bevidst: skjulte elementer der blot er usynlige i DOM'en, forbliver eventlyttere, hydrerer ved SSR og bidrager til JavaScript-bundle-evaluering. En betinget render fjerner elementet helt fra React-komponent-træet og eliminerer dermed al associeret overhead.

3. Databasedesign - implementeringsdetaljer

Chroniqos databaselag er implementeret med Prisma ORM (Object-Relational Mapper) (v7) mod en PostgreSQL-instans hostet på Neon.tech. Prisma-skemaet i `prisma/schema.prisma` er den eneste autoritative kilde til databasemodellen - DDL (Data Definition Language) genereres og vedligeholdes udelukkende af Prisma og skrives aldrig manuelt.

3.1. Forretningsregler implementeret direkte i databaselaget

En central designbeslutning er at placere centrale forretningsregler som database-constraints frem for udelukkende at håndhæve dem i applikationslaget. Fordelen er, at reglerne gælder uanset hvilken kodelinje der forsøger at skrive til databasen - de kan ikke omgås ved en fejlbehæftet service-implementering eller et direkte databasekald under debugging.

3.1.1. Dagsformens unikhedsbetingelse - `@@unique([userId, date])`

Kravet om maksimalt én dagsformsregistrering per bruger per kalenderdag er implementeret som en sammensat unikhedsbegrænsning direkte på `DailyStatus`-modellen:

```
1. // prisma/schema.prisma
2. model DailyStatus {
3.   id      String  @id @default(cuid())
4.   userId  String
5.   value   Int     // 0-4 matcher 1-5 skalaen
6.   note    String?
7.   date    DateTime @db.Date // Gemt som DATE, ikke TIMESTAMP
8.   createdAt DateTime @default(now())
9.   updatedAt DateTime @updatedAt
10.
11.   user User @relation(fields: [userId], references: [id], onDelete: Cascade)
12.
13.   messages Message[]
14.
15.   @@unique([userId, date])
16. }
```

Kodeudsnit 14: `DailyStatus`-modellen i `prisma/schema.prisma`. `@db.Date`-annotationen kortlægger feltet til PostgreSQL `DATE`-typen (udelukkende kalenderdag), og `@@unique([userId, date])` håndhæver maksimalt én registrering per bruger per dag på databaseniveau.

`@db.Date`-annotationen er kritisk: den instruerer Prisma i at kortlægge `date`-feltet til PostgreSQL's native `DATE`-type frem for `TIMESTAMP` (11). En `DATE`-kolonne gemmer udelukkende kalenderdag (YYYY-MM-DD) uden tidspunkt eller tidszone - to registreringer fra den samme bruger, om morgenen kl. 07:00 og aftenen kl. 22:00, vil begge have `date = '2026-05-13'` og dermed ramme den sammensatte unikhedsbegrænsning ved det andet forsøg.

Service-laget anvender `upsert`-semantik frem for `create`. Datoen normaliseres til UTC-midnat via hjælpefunktionen `getNormalizedDate()` inden selve kaldet - dette sikrer en konsistent datofortolkning på tværs af tidszoner og browsere, da en rå `Date()`-instans ville indeholde et lokalt tidsstempel der potentielt kortlægger til en anden kalenderdag i UTC. En eksisterende registrering for den normaliserede dato opdateres frem for at forårsage en databasefejl:

```
1. // src/services/daily-status.service.ts (uddrag)
2. export async function upsertDailyStatus(userId: string, data: DailyStatusDTO) {
3.   const targetDate = getNormalizedDate(data.date);
4.
5.   const status = await prisma.dailyStatus.upsert({
6.     where: { userId_date: { userId, date: targetDate } },
7.     update: { value: data.value, note: data.note || null },
8.     create: {
9.       userId,
10.      value: data.value,
11.      note: data.note || null,
12.      date: targetDate,
13.    },
14.  });
15.  return status;
16. }
```

Kodeudsnit 15: `upsertDailyStatus`-funktionen i `src/services/daily-status.service.ts`. Datoen normaliseres til UTC-midnat via `getNormalizedDate()` for at undgå tidszone-relaterede fortolkningsfejl på tværs af browsere og operativsystemer. Upsert-semantikken er idempotent: eksisterende registrering for den givne dato opdateres; ingen ny registrering oprettes. Kombineret med `@@unique([userId, date])`-constraintet i databaselaget elimineres race conditions ved samtidige `POST`-kald.

`upsert` er idempotent: det er ligegyldigt om det er første eller femte kald for dags dato - resultatet er altid én registrering med den seneste værdi. Kombinationen af `@@unique([userId, date])` og `upsert` giver en race-condition-fri implementering der håndterer samtidige kald korrekt.

3.1.2. GDPR-sletning via onDelete: Cascade

Kravspecifikationen i Bilag 1 (12) stiller krav om, at sletning af en brugerkonto sletter alt tilknyttet indhold. Dette er implementeret via `onDelete: Cascade` på samtlige relationer fra `User`-modellen (13):

```
1. // prisma/schema.prisma
2. model Post {
3.   id          String   @id @default(cuid())
4.   authorId    String
5.   // ...
6.
7.   author      User      @relation(fields: [authorId], references: [id], onDelete: Cascade)
8.   comments    Comment[]
9.   savedBy     SavedPost[]
10. }
11.
12. model Comment {
13.   id          String   @id @default(cuid())
14.   postId      String
15.   authorId    String
16.   parentId    String?
17.   // ...
18.
19.   post        Post      @relation(fields: [postId], references: [id], onDelete: Cascade)
20.   author      User      @relation(fields: [authorId], references: [id], onDelete: Cascade)
21.   parent      Comment? @relation("CommentReplies", fields: [parentId], references: [id], onDelete: Cascade)
22.   replies     Comment[] @relation("CommentReplies")
23. }
```

Kodeudsnit 16: *Post- og Comment-modellerne i `prisma/schema.prisma` med `onDelete: Cascade`-annotationer. Sletning af en bruger propagerer automatisk til alle opslag og kommentarer i en enkelt atomisk PostgreSQL-transaktion.*

`onDelete: Cascade` på `Post.author`-relationen betyder, at alle opslag slettes automatisk når brugeren slettes. `onDelete: Cascade` på `Comment.post`-relationen betyder, at alle kommentarer til et opslag slettes når opslaget slettes. Som en direkte kaskadeeffekt slettes både bruger, opslag og kommentarer i én atomisk transaktion af PostgreSQL's constraint-engine, uden at applikationslaget behøver at orchestrere rækkefølgen manuelt.

Et bevidst undtagelsestilfælde er GlobalBan:

```
1. // prisma/schema.prisma
2. model GlobalBan {
3.   id      String    @id @default(cuid())
4.   email   String    @unique
5.   userId  String?   @unique
6.   // ...
7.
8.   user User? @relation(fields: [userId], references: [id], onDelete: SetNull)
9. }
```

Kodeudsnit 17: GlobalBan-modellen med onDelete: SetNull. Bandlysningen overlever kontosletningen ved at bevare email-feltet og sætte userId til null - dette forhindrer omgåelse af forbud via kontosletning og genoprettelse.

onDelete: SetNull på GlobalBan.user betyder, at en ban-registrering overlever kontosletningen: userId sættes til null, men email-feltet bevares. Dette er en bevidst designbeslutning: en bruger der er bandlyst og derefter sletter sin konto for at omgå forbuddet, kan ikke oprette en ny konto med samme e-mailadresse, fordi email-feltet forbliver i GlobalBan-tabellen med et aktivt ban.

3.1.3. Sammensatte primærnøgler som forretningsregel - `@@id([userId, postId])`

Koblingstabeller, der modellerer mange-til-mange-relationer, anvender sammensatte primærnøgler frem for surrogatnøgler (interne, unikke identifikatorer uafhængige af tabellens reelle data). Mønsteret er konsekvent anvendt på tværs af alle rene koblingstabeller:

```
1. // prisma/schema.prisma
2. model PostSupport {
3.   userId    String
4.   postId    String
5.   createdAt DateTime @default(now())
6.
7.   user User @relation(fields: [userId], references: [id], onDelete: Cascade)
8.   post Post @relation(fields: [postId], references: [id], onDelete: Cascade)
9.
10.  @@id([userId, postId])
11. }
12.
13. model MessageReaction {
14.   userId    String
15.   messageId String
16.   emoji     String
17.   createdAt DateTime @default(now())
18.
19.   user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)
20.   message Message @relation(fields: [messageId], references: [id], onDelete: Cascade)
21.
22.  @@id([userId, messageId])
23. }
```

Kodeudsnit 18: PostSupport- og MessageReaction-modellerne i prisma/schema.prisma som repræsentative eksempler på det sammensatte primærnøgle-mønster. Mønsteret anvendes identisk på CommentSupport, SavedPost, HiddenPost, HiddenCommunity, BlockedCommunity, Friendship og RecentPost.

`@@id([userId, postId])` håndhæver, at én bruger maksimalt kan støtte ét opslag én gang - og tilsvarende for MessageReaction, CommentSupport, SavedPost, HiddenPost og Friendship. Forsøg på at indsætte en dublet returnerer en PostgreSQL `unique_violation`-fejl (kode 23505), som applikationslaget håndterer og mapper til en HTTP 409 Conflict.

Den sammensatte primærnøgle er ikke blot en unikhedsbegrænsning - den fungerer også som et effektivt b-træ-indeks på begge kolonner. Dette accelererer opslag som "har bruger X støttet opslag Y?" uden et separat `@@index`-direktiv.

3.1.4. GlobalBan-persistering og e-mail-baseret blokering

GlobalBan-modellen håndterer en edge case der ikke kan løses via bruger-ID-baseret unikhed alene.

Den fulde model fremgår af Kodeudsnit 19:

```
1. // prisma/schema.prisma
2. model GlobalBan {
3.   isDummy      Boolean    @default(false)
4.   isActive     Boolean    @default(true)
5.   id           String     @id @default(cuid())
6.   email        String     @unique
7.   userId       String?    @unique
8.   reason       String?
9.   expiresAt    DateTime?
10.  deleteToken  String?    @unique
11.  createdAt    DateTime   @default(now())
12.  updatedAt    DateTime   @updatedAt
13.
14.  user User? @relation(fields: [userId], references: [id], onDelete: SetNull)
15. }
```

Kodeudsnit 19: Den fulde GlobalBan-model i prisma/schema.prisma. Kombinationen af email @unique (non-nullable, aldrig slettet) og userId? @unique (nullable, onDelete: SetNull) implementerer e-mail-baseret bandlysningspersistering der overlever kontosletning.

email @unique og userId? @unique er begge unikhedsbegrænsninger, men med forskellig semantik: userId er nullable og sættes til null ved kontosletning (onDelete: SetNull), mens email er non-nullable og aldrig slettes. Resultatet er, at en bandlyst bruger, der sletter sin konto i håbet om at oprette en ny, vil opleve, at tilmeldingsflowet afviser den pågældende e-mailadresse, da GlobalBan-tabellen fortsat indeholder en aktiv GlobalBan-række for den pågældende e-mail.

expiresAt-feltet understøtter tidsbegrænsede bandlysninger. isActive-feltet muliggør, at en bandlysning markeres som inaktiv uden at slette historikken. deleteToken er en engangstoken der genereres når en bandlyst bruger forsøger at slette sin konto via den normale sletningsflow - tokenet bruges til at sætte userId til NULL korrekt.

3.2. Migrerings- og databasehostingsstrategi

Databaselaget hviler på tre beslutninger med direkte konsekvenser for udviklingsprocessen og produktionsinfrastrukturen: valget af `prisma db push` som skemasynkroniseringsstrategi i MVP-fasen (afsnit 3.2.1), Neon.techs serverless connection pooling som løsning på serverless-infrastrukturens forbindelsesudfordringer (afsnit 3.2.2), og Prisma v7's opsplittede konfigurationsmodel der adskiller skemastruktur fra miljøspecifik runtime-konfiguration (afsnit 3.2.3).

3.2.1. prisma db push vs. prisma migrate - begrundelse for MVP-valget

Prisma understøtter to skemasynkroniseringsstrategier med fundamentalt forskellig semantik (14):

Tabel 6: Sammenligning af `prisma db push` og `prisma migrate dev` - de to primære skemasynkroniseringsstrategier i Prisma ORM med egenskaber, fordele og begrænsninger.

Sammenligning af prisma db push og prisma migrate dev		
Egenskab	prisma db push	prisma migrate dev
Genererer migrationsfiler	Nej	Ja
Ideel til	Rapid prototyping, solo-MVP	Team-udvikling, produktions-systemer
Rollback-mulighed	Nej (manuelt)	Ja (via migrationsfiler)
Historik	Ingen	Fuld migreringshistorik
Destruktive ændringer	Advarer, men gennemfører	Kræver eksplicit bekræftelse
CI/CD-egnet	Ikke ideel	Ja (<code>prisma migrate deploy</code>)

For et soloprojekt i MVP-fasen er `prisma db push` valgt på baggrund af to faktorer:

1. Skemaet ændrede sig hyppigt i de tidlige iterationer, og den overhead, der følger med at navngive og gennemgå en migrationsfil ved hver skemaændring, ville have bremset iterationshastigheden.
2. Med én enkelt udvikler og ingen parallelle branches er risikoen for migrationskonflikter ikke-eksisterende.

Trade-offet er klart: `prisma db push` efterlader ingen migreringshistorik, og en destruktiv skemaændring (f.eks. sletning af en kolonne) kan ikke rulles tilbage automatisk. Ved overgang til teamudviklingsfase skal strategien skiftes til `prisma migrate dev` med en baseline-migrering, der afspejler det nuværende skema (14).

3.2.2. Neon.tech - serverless connection pooling

Next.js App Router deployet på Vercel kører som serverless funktioner: hver API-forespørgsel kan eksekveres i en ny funktion-instans, der etablerer sin egen databaseforbindelse. PostgreSQL-servere har en maksimal grænse for samtidige forbindelser (`max_connections`), og naive serverless-deployments udtømmer denne grænse hurtigt ved moderat trafik.

Neon.tech løser dette via to mekanismer:

1. **Serverless driver med HTTP-transport:** Neon tilbyder en serverless PostgreSQL-driver (`@neondatabase/serverless`) der tunneler forespørgsler over HTTP/WebSocket frem for en persistent TCP-forbindelse. HTTP-forbindelser lukkes øjeblikkeligt efter forespørgslens svar og belaster ikke `max_connections`.
2. **Connection pooler-endpoint:** Forbindelsesstrengen i `DATABASE_URL` peger på Neons connection pooler-endpoint (`-pooler`-suffix i hostname), der anvender PgBouncer internt til at multipleksere mange applikationsforbindelser over et begrænset antal faktiske PostgreSQL-forbindelser (15).

3.2.3. Miljøvariabelhåndtering og Prisma v7-konfiguration

Prisma v7 introducerede en opsplitning af konfiguration og skema: frem for at definere databaseforbindelsen direkte i `schema.prisma` specificeres den i en separat `prisma.config.ts`-fil via `datasource.url`. I lokalt udviklingsmiljø indlæses `DATABASE_URL` fra `.env.local` via `dotenv`; i CI/CD-miljøet injiceres variabelen direkte af runneren. Resultatet er en ren adskillelse mellem skemastruktur og miljøspecifik konfiguration:

```
1. // prisma.config.ts
2. import dotenv from "dotenv";
3. import { defineConfig, env } from "prisma/config";
4.
5. // Load .env.local locally; in CI the variable is injected by the runner
6. if (!process.env.CI) {
7.   dotenv.config({ path: ".env.local" });
8. }
9.
10. export default defineConfig({
11.   schema: "prisma/schema.prisma",
12.   migrations: {
13.     path: "prisma/migrations",
14.   },
15.   datasource: {
16.     url: env("DATABASE_URL"),
17.   },
18. });
```

Kodeudsnit 20: `prisma.config.ts` - Prisma v7's adskilte konfigurationsfil. `DATABASE_URL` hentes fra procesmiljøet via `env()` og injiceres som `datasource.url`. I lokalt udviklingsmiljø indlæses variabelen fra `.env.local` via `dotenv`; i CI/CD-miljøet (GitLab, Vercel) injiceres variabelen direkte af runneren. `schema.prisma`-filen indeholder ingen direkte reference til databaseforbindelsen - adskillelsen sikrer, at skemaet kan versionstyres og deles frit. Neons connection pooling aktiveres via `pooler-endpointet` i selve `DATABASE_URL`-forbindelsessstrengen.

Adskillelsen er bevidst: `schema.prisma` er en ren strukturbeskrivelse af databasemodellen uden miljøspecifikke referencer, mens `prisma.config.ts` håndterer den runtime-afhængige konfiguration. `schema.prisma`-filen kan dermed versionstyres og deles frit, mens `prisma.config.ts` er afhængig af miljøvariabler der aldrig committes til repositoriet.

4. Softwaredesign - implementeringsdetaljer

Dette afsnit dokumenterer implementeringsdetaljerne bag Chroniqos serverside-arkitektur. Strukturen følger lagdelingen fra Route Handler-laget via service-laget til databaselaget, og dækker autentificering, rate limiting, AI-integration og chat-implementeringen.

4.1. Monorepostruktur og komponentarkitektur

Applikationens lagdelte arkitektur er bygget op om tre sammenhængende principper: Route Handlers som tynde controllers der udelukkende autentificerer, validerer og delegerer (afsnit 4.1.1), et servicelaget der centraliserer al forretningslogik og muliggør isoleret enhedstest (afsnit 4.1.2), og et DTO-lag baseret på Zod-skemaer der håndhæver over-posting-beskyttelse og sikrer end-to-end type-sikkerhed fra HTTP-krop til Prisma-kald (afsnit 4.1.3).

4.1.1. Route Handler-arkitektur - tynde controllers og Zod-validering

Next.js App Router's Route Handlers (`src/app/api/**`) fungerer som applikationens HTTP-grænseflade. Designprincippet er konsekvent tynde controllers: en Route Handler har udelukkende ansvar for at:

1. Autentificere kalderen
2. Validere input med Zod
3. Delegere til service-laget

Al forretningslogik placeres i servicelaget og aldrig i handleren selv.

`src/app/api/posts/route.ts` illustrerer mønsteret som eksempel:

```
1. // src/app/api/posts/route.ts
2. export async function POST(request: Request) {
3.   // 1. Autentificering
4.   const session = await auth();
5.   if (!session?.user?.id) {
6.     return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
7.   }
8.
9.   // 2. Zod-validering - over-posting forhindres her
10.  const body = await request.json();
11.  const parsedData = createPostSchema.safeParse(body);
12.  if (!parsedData.success) {
13.    return NextResponse.json(
14.      { error: "Validation failed", details: parsedData.error.issues },
15.      { status: 400 },
16.    );
17.  }
18.
19.  // (Mute-tjek er en forretningsregel tilknyttet dette endpoint - se handler for detaljer)
20.
21.  // 3. Delegation til service-laget
22.  const post = await createPost(session.user.id, parsedData.data);
23.  return NextResponse.json({ message: "Post created successfully", post }, { status: 201 });
24. }
```

Kodeudsnit 21: Route Handler-mønsteret i `src/app/api/posts/route.ts`. Handleren autentificerer via `auth()`, validerer med `createPostSchema.safeParse()` og delegerer til `createPost()` i service-laget. Handleren indeholder ingen forretningslogik ud over mute-tjekket, der er endpoint-specifikt og kræver direkte database-adgang.

`safeParse` frem for `parse` er bevidst: `parse` kaster en undtagelse (exception) ved valideringsfejl, mens `safeParse` returnerer et resultatobjekt med en `success`-boolean. Dette giver en præcis HTTP 400 med detaljer om valideringsfejlene frem for en generisk 500 fra en ubehandlet undtagelse.

4.1.2. Service-lag og forretningslogik

Service-filerne i `src/services/` rummer al domænelogik. Adskillelsen fra Route Handleren muliggør, at services kan testes med Jest uden at skulle simulere HTTP-kontekst - en direkte konsekvens af lagdelingen, der er dokumenteret i testresultaterne i Bilag 7 (3).

`createPost` i `src/services/post.service.ts` illustrerer service-lagets ansvarsområde:

```
1. // src/services/post.service.ts (uddrag)
2. export async function createPost(userId: string, data: CreatePostDTO) {
3.   // Forretningsregel: anonym opslag kræver et fællesskab som destination
4.   if (data.isAnonymous && !data.communityId) {
5.     throw new Error("Cannot post anonymously to your personal profile");
6.   }
7.
8.   // Forretningsregel: private fællesskaber kræver accepteret medlemskab
9.   if (data.communityId) {
10.    const membership = await prisma.communityMember.findUnique({
11.      where: { userId_communityId: { userId, communityId: data.communityId } },
12.    });
13.    const community = await prisma.community.findUnique({
14.      where: { id: data.communityId },
15.    });
16.    if (community?.isPrivate && membership?.status !== "ACCEPTED") {
17.      throw new Error("You must be an accepted member of this private community to post");
18.    }
19.  }
20.
21.  // Afstemning af afstemningsmetadata: nulstil stemmer, beregn udløb, valider optioner
22.  if (data.type === "poll") {
23.    // ... (se fuld implementering i post.service.ts)
24.  }
25.
26.  return prisma.post.create({ data: { /* ... */ } });
27. }
```

Kodeudsnit 22: `createPost`-funktionen i `src/services/post.service.ts` med forretningsreglerne for anonymt opslag og privat fællesskabsadgang. Servicefunktioner kaster undtagelser (exceptions); Route Handleren håndterer mapping til HTTP-statuskoder.

Bemærk, at `createPost` kaster undtagelser (exceptions) frem for at returnere fejlobjekter. Route Handleren fanger undtagelserne i en `try/catch` og mapper dem til de korrekte HTTP-statuskoder. Denne konvention er konsistent på tværs af alle services og giver en klar kontrakt: services signalerer fejl med undtagelser; handlers oversætter dem til HTTP.

Afstemninger (`poll`) er et eksempel på metadata-normalisering i service-laget: klientens `metadata`-objekt valideres strukturelt af Zod i DTO-laget (se 4.1.3), men semantiske forretningsregler (antal optioner, tekst-maksimumslængde, initialisering af `voters = {}` og `totalVotes = 0`, beregning af `closesAt`) håndhæves udelukkende i service-laget. Dette forhindrer, at en klient kan manipulere afstemningsresultater ved at sende et `metadata`-objekt med præudfyldte stemmedata.

4.1.3. DTO-lag og over-posting-beskyttelse

Data Transfer Objects (DTOer) er implementeret med Zod i `src/lib/dtos/**` og definerer den præcise kontrakt for hvad klienten må sende til et givet endpoint. `createPostSchema` eksemplificerer over-posting-beskyttelsen:

```
1. // src/lib/dtos/post.dto.ts
2. export const createPostSchema = z.object({
3.   title:      z.string().min(1).max(300),
4.   type:       z.enum(["text", "image", "video", "youtube", "link", "poll"]).default("text"),
5.   communityId: z.string().nullable().optional(),
6.   content:    z.string().optional(),
7.   metadata:   z.record(z.string(), z.unknown()).optional().nullable(),
8.   isAnonymous: z.boolean().default(false),
9. });
10.
11. export type CreatePostDTO = z.infer<typeof createPostSchema>;
```

Kodeudsnit 23: `createPostSchema` i `src/lib/dtos/post.dto.ts`. Skemaet definerer den eksplicitte whitelist af accepterede felter. `z.infer<typeof createPostSchema>` genererer `CreatePostDTO`-typen der bruges som parameter i service-laget og giver end-to-end type-sikkerhed fra HTTP-krop til Prisma-kald.

Skemaet definerer præcis hvilke felter der accepteres. Felter som `authorId`, `viewCount`, og `isDummy` er fraværende i skemaet - selv om en klient sender dem i request-body'en, vil `safeParse` ignorere ukendte nøgler (Zods standardadfærd med `.strip()`). Et scenarie hvor en klient forsøger at sende `{ "authorId": "admin-id", "isDummy": false }` for at udgive sig for en anden bruger, fanges dermed på valideringslaget: kun de felter der eksplicit er defineret i skemaet, når frem til service-laget.

`CreatePostDTO` er TypeScript-typen der er udledt direkte fra skemaet via `z.infer<>`. Denne type bruges som parameter-type i service-funktioner, så TypeScript-compileren garanterer typesikkerhed end-to-end fra HTTP-kroppen til Prisma-kaldet.

4.2. Autentificering - implementeringsdetaljer

Autentificeringslaget er konfigureret i `src/auth.ts` og dækker fire implementeringsaspekter: provider-konfigurationen og JWT-strategiens grundlag (afsnit 4.2.1), `signIn`-callbackets rolle som primær adgangsvagt med ban-tjek og OAuth-verifikation (afsnit 4.2.2), JWT- og session-callbackenes berigelse og synkronisering af token-data (afsnit 4.2.3), samt den bevidste trade-off bag `allowDangerousEmailAccountLinking` (afsnit 4.2.4).

4.2.1. Auth.js-konfiguration (src/auth.ts) - overblik

src/auth.ts er det centrale konfigurationspunkt for Auth.js (NextAuth v5). Filen eksporterer handlers, signIn, signOut og auth - de fire primære API-overflader, der bruges på tværs af hele applikationen. Konfigurationen specificerer:

- **To providers:** Google OAuth 2.0 og Credentials (e-mail/adgangskode)
- **Session-strategi:** JWT (påkrævet ved brug af Credentials-provider)
- **Tre callbacks:** signIn, jwt og session - eksekveret i den nævnte rækkefølge

```
1. // src/auth.ts (uddrag - provider-konfiguration)
2. export const { handlers, signIn, signOut, auth } = NextAuth({
3.   adapter: PrismaAdapter(prisma),
4.   session: { strategy: "jwt" },
5.   providers: [
6.     Google({
7.       clientId: process.env.GOOGLE_CLIENT_ID,
8.       clientSecret: process.env.GOOGLE_CLIENT_SECRET,
9.       allowDangerousEmailAccountLinking: true,
10.      authorization: { params: { prompt: "select_account" } },
11.    }),
12.    CredentialsProvider({
13.      name: "Credentials",
14.      credentials: {
15.        email: { label: "Email", type: "email" },
16.        password: { label: "Password", type: "password" },
17.      },
18.      async authorize(credentials) {
19.        if (!credentials?.email || !credentials?.password) return null;
20.
21.        const user = await prisma.user.findUnique({
22.          where: { email: credentials.email as string },
23.        });
24.
25.        // Dummy-konti blokeres fra login - de er kun til testdatagenerering
26.        if (!user || !user.hashedException || user.isDummy) return null;
27.
28.        const passwordsMatch = await bcrypt.compare(
29.          credentials.password as string,
30.          user.hashedException,
31.        );
32.
33.        return passwordsMatch ? { id: user.id, /* ... */ } : null;
34.      },
35.    }),
36.  ],
37.  // callbacks: se 4.2.2 og 4.2.3
38.});
```

Kodeudsnit 24: Provider-konfigurationen i src/auth.ts. Google-provideren anvender allowDangerousEmailAccountLinking: true og prompt: "select_account" (se 4.2.4). authorize-funktionen i Credentials-provideren blokerer eksplicit dummy-konti fra login ved at returnere null hvis isDummy er true.

JWT-strategien er et krav, der følger direkte af Credentials-providerens natur: Credentials-baseret login er per Auth.js-design uforeneligt med databasesessioner, da der ikke er noget OAuth-token, der kan bruges som sessionsanker (16). JWT lagres som en `HttpOnly`-cookie, der er utilgængelig fra JavaScript og dermed immun over for XSS-angreb (17).

4.2.2. signIn-callbacket som primær adgangsvagt

signIn-callbacket eksekveres inden Auth.js udsteder en session og er det centrale sted for adgangskontrol ved login. Callbacket håndterer to ansvarsområder:

4.2.2.1. Global ban-tjek

```
1. // src/auth.ts - signIn callback
2. async signIn({ user, account }) {
3.   if (user.email) {
4.     const activeBan = await prisma.globalBan.findFirst({
5.       where: {
6.         email: user.email,
7.         isActive: true,
8.         OR: [{ expiresAt: null }, { expiresAt: { gt: new Date() } }],
9.       },
10.    });
11.
12.    if (activeBan) {
13.      // Generer engangstoken til datahåndtering uden aktiv session
14.      const token = crypto.randomUUID();
15.      await prisma.globalBan.update({
16.        where: { id: activeBan.id },
17.        data: { deleteToken: token },
18.      });
19.
20.      const params = new URLSearchParams();
21.      params.set("banned", "true");
22.      params.set("token", token);
23.      params.set("dataAlreadyDeleted", String(Boolean(activeBan.userId === null)));
24.      if (activeBan.reason) params.set("reason", activeBan.reason);
25.      if (activeBan.expiresAt) params.set("expires", activeBan.expiresAt.toISOString());
26.
27.      // OAuth bruger popup-flow - data sendes via postMessage til åbnervinduet
28.      if (account?.provider !== "credentials") {
29.        return `/api/auth/popup-callback?${params.toString()}`;
30.      }
31.
32.      return `/login?${params.toString()}`;
33.    }
34.  }
```

Kodeudsnit 25: Bandlysningstjekket i signIn-callbacket. GlobalBan.findFirst anvender en sammensat OR-betingelse til at håndtere både permanente og tidsbegrænsede bandlysninger. deleteToken genereres med crypto.randomUUID() og giver den bandlyste bruger mulighed for at slette sine data uden en aktiv session.

En aktiv bandlysning defineres som en GlobalBan-række, hvor isActive = true og enten expiresAt er null (permanent) eller expiresAt > now() (tidsbegrænset bandlysning, som endnu ikke er udløbet). Bandlysningen bruger redirects til loginsiden med bandlysningsdetaljer som query parametre - men aldrig direkte til en beskyttet side.

At redirect-URL'en returneres som en streng frem for false er intentionelt: false ville vise Auth.js' generiske fejlside, mens en streng-URL giver fuld kontrol over brugeroplevelsen under ban-notifikationen.

4.2.2.2. OAuth signup-verifikation

```
1. // src/auth.ts - signIn callback
2. // OAuth brugere springes over den manuelle e-mailbekræftelse
3. if (account?.provider !== "credentials" && user.id) {
4.   await prisma.user.updateMany({
5.     where: { id: user.id, signupVerified: null },
6.     data: { signupVerified: new Date() },
7.   });
8. }
9.
10. return true;
11. },
```

Kodeudsnit 26: OAuth signup-verifikationslogikken. `updateMany` med `signupVerified: null`-filteret er idempotent og er en no-op (no operation) ved efterfølgende logins.

`updateMany` med `signupVerified: null`-filteret er en no-op (no operation) på efterfølgende logins: kun den allerførste OAuth-login sætter `signupVerified`; efterfølgende kald finder ingen rækker der opfylder { `signupVerified: null` } og gør ingenting. Credentials-brugere gennemgår den manuelle e-mail-bekræftelses-flow og udelades via `provider !== "credentials"`-tjekket.

4.2.3. JWT- og session-callbacks

jwt-callbacket har ansvar for at berige JWT'ens payload med applikationsspecifikke felter, der ikke er til stede i Auth.js' standard-token:

```
1. // src/auth.ts - signIn callback
2. async jwt({ token, user, trigger, session }) {
3.   // Kun ved det første login - user-objektet er kun tilgængeligt her
4.   if (user) {
5.     token.id = user.id;
6.     token.username = user.username;
7.     token.onboarded = user.onboarded;
8.     token.hasPassword = user.hasPassword ?? !!user.hashPassword;
9.     token.image = user.image;
10.    token.avatarEmoji = user.avatarEmoji;
11.    token.avatarBgColor = user.avatarBgColor;
12.    token.role = user.role;
13.    token.emailVerified = user.emailVerified;
14.  }
15.
16.  // Klientsiden kan opdatere tokenet via useSession().update({ onboarded: true })
17.  if (trigger === "update" && session) {
18.    if (session.onboarded !== undefined) token.onboarded = session.onboarded;
19.    if (session.username !== undefined) token.username = session.username;
20.    if (session.hasPassword !== undefined) token.hasPassword = session.hasPassword;
21.    if (session.image !== undefined) token.image = session.image;
22.    if (session.avatarEmoji !== undefined) token.avatarEmoji = session.avatarEmoji;
23.    if (session.avatarBgColor !== undefined) token.avatarBgColor = session.avatarBgColor;
24.    if (session.role !== undefined) token.role = session.role;
25.  }
26.
27.  return token;
28. },
```

Kodeudsnit 27: jwt-callbacket i src/auth.ts. if (user)-grenen eksekveres udelukkende ved det første login og beriger JWT-payloaden med applikationsspecifikke felter - herunder token.image der persisterer profilbilledet på tværs af sessioner. trigger === "update"-grenen håndterer klientinitierede token-opdateringer via useSession().update(), der bruges af onboarding-flowet og brugerindstillingssiderne til at synkronisere token-data uden re-login.

trigger === "update"-grenen håndterer klientinitierede token-opdateringer. Onboarding-flowet bruger dette til at skrive onboarded: true til JWT'en umiddelbart efter at brugeren fuldfører onboarding, uden at kræve et nyt login (16). Tilsvarende opdateres username og avatarEmoji når brugeren ændrer dem i indstillingerne - JWT-data holdes dermed i sync med databasen uden at afvente token-rotation.

session-callbacket kopierer de relevante JWT-felter til det klienttilgængelige session-objekt.

Next.js' TypeScript-moduludvidelse i toppen af auth.ts (declare module "next-auth") udvider Session-interfacet med disse felter, så session.user.role, session.user.username m.fl. er typesikre på klientsiden.

4.2.4. allowDangerousEmailAccountLinking - bevidst trade-off

Google-provideren er konfigureret med `allowDangerousEmailAccountLinking: true`. Denne indstilling tillader, at et eksisterende credentials-baseret konto automatisk kobles til et Google OAuth 2.0-konto, hvis begge deler den samme e-mailadresse.

Standardadfærden i Auth.js er at afvise dette link og kaste en fejl, da en angriber der kontrollerer en OAuth-udbyder potentielt kan kapre en eksisterende konto ved at registrere en Google-konto med offerets e-mailadresse. Risikoen er reel men betinget af, at angriberen også kontrollerer offerets Google-konto eller kan oprette en Google-konto med en e-mailadresse, Google normalt verificerer.

Valget er truffet på baggrund af brugervenlighed: en bruger der tilmelder sig med e-mail og siden forsøger at logge ind med Google (en hyppig brugeradfærd) ville uden denne indstilling møde en uforklarlig fejl. For en MVP-plattform i beta-fasen vurderes brugervenlighedsgevinsten at overstige risikoprofilen. Ved overgang til produktion med sensitive brugerdata bør denne indstilling revurderes og potentielt erstattes af et eksplicit konto-koblingsflow.

4.3. Rate limiting og Edge Middleware - implementeringsdetaljer

Edge Middleware-laget varetager tre sikkerhedsrelaterede ansvarsområder: den fulde proxy-funktion i `src/proxy.ts` der håndterer rate limiting, API-bypass og locale-routing i prioriteret rækkefølge (afsnit 4.3.1), sliding-window-algoritmens konfiguration og den differentierede opdeling af auth- og Gemini-limiters (afsnit 4.3.2), og endelig den Redis-baserede realtids-banvalidering der omgør JWT-tokens iboende manglende tilbagekaldelighed (afsnit 4.3.3).

4.3.1. Edge Middleware (src/proxy.ts) - fuldt kodeudsnit

`src/proxy.ts` eksporteres som Next.js Middleware og eksekveres i Vercels Edge Runtime - tættere på brugeren geografisk og inden applikationskoden overhovedet når serverless funktions-laget.

Filen varetager tre ansvarsområder i prioriteret rækkefølge:


```
1. // src/proxy.ts
2. export async function proxy(request: NextRequest) {
3.   const { pathname } = request.nextUrl;
4.
5.   // -- 1. Edge Rate Limiting for Auth Routes (Fail Fast) -----
6.   if (pathname.startsWith("/api/auth")) {
7.     const ip =
8.       request.headers.get("x-forwarded-for") ??
9.       request.headers.get("x-real-ip") ??
10.      "127.0.0.1";
11.
12.     try {
13.       const { success, limit, reset, remaining } = await ratelimiter.limit(ip);
14.
15.       if (!success) {
16.         return NextResponse.json(
17.           { error: "Too Many Requests" },
18.           {
19.             status: 429,
20.             headers: {
21.               "X-RateLimit-Limit": limit.toString(),
22.               "X-RateLimit-Remaining": remaining.toString(),
23.               "X-RateLimit-Reset": reset.toString(),
24.             },
25.           },
26.         );
27.       }
28.     } catch (error) {
29.       // Graceful degradation: Redis utilgængeligt → request tillades
30.       console.error("[Proxy] Rate limiting error:", error);
31.     }
32.   }
33.
34.   // -- 2. Bypass locale-routing for API-ruter -----
35.   if (pathname.startsWith("/api")) {
36.     return NextResponse.next();
37.   }
38.
39.   // -- 3. Locale-routing for ikke-API-ruter -----
40.   const pathnameHasLocale = locales.some(
41.     (locale) => pathname.startsWith(`/${locale}/`) || pathname === `/${locale}`,
42.   );
43.
44.   if (pathnameHasLocale) {
45.     const response = NextResponse.next();
46.     response.headers.set("x-url", request.url);
47.     return response;
48.   }
49.
50.   const locale = getLocale(request);
51.   request.nextUrl.pathname = `/${locale}${pathname}`;
52.   const response = NextResponse.redirect(request.nextUrl);
53.   response.cookies.set("NEXT_LOCALE", locale, { path: "/" });
54.   return response;
55. }
56.
57. export const config = {
58.   matcher: ["/((?!_next/static|_next/image|favicon.ico|.*\\.svg$|sounds).*)"],
59. };
```

Kodeudsnit 28: Den fulde `proxy`-funktion i `src/proxy.ts`. De tre ansvarsområder eksekveres i prioriteret rækkefølge: `auth rate limiting`, dernæst `API bypass` og dernæst `locale-routing`. `catch`-blokken implementerer `graceful degradation`: `Redis-fejl` resulterer i en advarsel i konsollen, ikke en `429-respons` til brugeren.

Rate limiting er begrænset til `/api/auth`-ruter (fail fast-princippet): disse er de mest attraktive mål for credential stuffing og brute force, og det er unødvendigt at belaste Upstash Redis med rate-limit-opslag for hvert enkelt API-kald i applikationen.

`graceful degradation` i `catch`-blokken er en bevidst sikkerhedsafvejning: hvis Redis er utilgængeligt, tillades den gældende request at fortsætte frem for at returnere en HTTP 500. Et Redis-nedbrud bør ikke resultere i, at login er umuligt for alle brugere. Alternativet ville være at afvise alle auth-requests ved Redis-nedbrud (fail closed), der er mere sikkert men langt mere skadeligt for tilgængelighed.

`x-url`-headeren der sættes ved locale-routing, videregiver den fulde URL (inkl. `query-parametre`) til server-komponenterne i layoutet, der bruger den til at opbygge præcise `callbackUrl`-redirects uden at miste `query-parametre`.

4.3.2. Sliding-window-algoritme og differentieret rate limiting

`src/lib/upstash.ts` konfigurerer to separate rate limiters med forskellig granularitet:

```
1. // src/lib/upstash.ts
2. export const redis = Redis.fromEnv();
3.
4. // Auth-limiter: 20 requests per IP per 10 sekunder
5. export const rateLimiter = new Ratelimit({
6.   redis,
7.   limiter: Ratelimit.slidingWindow(20, "10 s"),
8.   analytics: true,
9.   prefix: "@upstash/ratelimit",
10. });
11.
12. // Gemini-limiter: 20 requests per bruger per time
13. export const rateLimiterPerHour = new Ratelimit({
14.   redis,
15.   limiter: Ratelimit.slidingWindow(20, "1 h"),
16.   analytics: true,
17.   prefix: "@upstash/ratelimit/gemini",
18. });
```

Kodeudsnit 29: `rateLimiter` og `rateLimiterPerHour` i `src/lib/upstash.ts`. Begge anvender `Ratelimit.slidingWindow` med `analytics: true`, der giver Upstash-dashboardet historiske data om rate limit hits. De adskilte `prefix`-værdier sikrer, at de to limiters' Redis-nøgler ikke kolliderer.

Sliding-window-algoritmen er valgt frem for fixed window fordi den forhindrer boundary bursts: med en fixed-window-limiter på 20 requests per 10 sekunder kan en angriber sende 20 requests i sekund 9 og 20 requests i sekund 11 - 40 requests inden for et 2-sekunders vindue. Sliding-window-algoritmen beregner antallet af requests i det præcise vindue fra `now - 10s` til `now` og forhindrer dette mønster (18).

De to limiters er differentieret på to akser:

Tabel 7: Konfigurationsparametre for de to Upstash-rate limiters i `src/lib/upstash.ts` - nøgletype, vinduesstørrelse, grænse, formål og Redis-prefix.

Konfigurationsparametre for de to Upstash rate limiters i upstash.ts		
	rateLimiter (auth)	rateLimiterPerHour (Gemini)
Nøgle	IP-adresse	Bruger-ID
Vindue	10 sekunder	1 time
Grænse	20 requests	20 requests
Formål	Brute force / credential stuffing	API-omkostningsbegrænsning
Prefix	@upstash/ratelimit	@upstash/ratelimit/gemini

Auth-limiteren bruger IP-adresse som nøgle, da uautoriserede brugere ikke har et bruger-ID. Gemini-limiteren bruger bruger-ID som nøgle for at begrænse per-bruger API-forbrug og forhindre, at én bruger udtømmer den gratis Gemini API-kvote for hele platformen (18).

4.3.3. Realtids-banvalidering via Redis - setBanFlag og clearBanFlag

Et grundlæggende problem ved JWT-baserede sessioner er, at et JWT ikke kan tilbagekaldes: når en admin bandlyser en bruger, har den pågældende bruger stadig et gyldigt JWT i sin `HttpOnly`-cookie og kan fortsætte med at bruge applikationen indtil tokenet udløber naturligt.

`setBanFlag` og `clearBanFlag` i `upstash.ts` løser dette:

```
1. // src/lib/upstash.ts
2. export async function setBanFlag(userId: string): Promise<void> {
3.   await redis.set(`banned:${userId}`, "1", { ex: BAN_FLAG_TTL_SECONDS });
4. }
5.
6. export async function clearBanFlag(userId: string): Promise<void> {
7.   await redis.del(`banned:${userId}`);
8. }
```

Kodeudsnit 30: `setBanFlag` og `clearBanFlag` i `src/lib/upstash.ts`. `redis.set(key, "1", { ex: BAN_FLAG_TTL_SECONDS })` sætter en TTL-styret nøgle. `BAN_FLAG_TTL_SECONDS` er defineret i `src/lib/constants.ts` og matcher den konfigurerede maksimale JWT-levetid.

Når en admin bandlyser en bruger, kalder admin-service-laget `setBanFlag(userId)`, der skriver nøglen `banned:<userId>` til Redis med en TTL svarende til sessionens maksimale levetid (`BAN_FLAG_TTL_SECONDS`, defineret i `src/lib/constants.ts`). TTL'en sikrer, at Redis-nøglen automatisk ryddes op, selv om `clearBanFlag` aldrig kaldes.

Klientsiden poller `/api/auth/banned-account` med et kort interval via `ban-watcher.tsx` (`src/components/auth/ban-watcher.tsx`). Denne Route Handler tjekker Redis for nøglen `banned:<session.user.id>` og returnerer `{ banned: true }` hvis den eksisterer. Klienten afmelder brugeren og omdirigerer til loginsiden med `ban`-parametre i URL'en.

Mønsteret løser JWT-problemet uden at kræve database-polling fra middleware: Redis-opslaget er et sub-millisekund-kald der sker i klienten, ikke i Edge Runtime, og belaster dermed ikke auth-flowet for ikke-bandlyste brugere.

4.4. Niqo AI-integration - implementeringsdetaljer

Niqo-integrationen er bygget op om tre lag: et server-side PII-filter, der renser brugerinput, inden det forlader systemet (afsnit 4.4.1), en sessionshåndtering baseret på Redis TTL og PostgreSQL-persistering (afsnit 4.4.2), og et to-trins Gemini API-flow, der adskiller chat-oprettelse fra AI-svaret for at bevare en responsiv brugeroplevelse (afsnit 4.4.3).

4.4.1. PII-filter - regex-mønstre og kendte begrænsninger

`src/lib/utils/pii-filter.ts` implementerer en server-side anonymisering der fjerner Personhenførbare Oplysninger (PII) fra brugerinput, inden teksten sendes til Google Gemini API.

Filteret anvender fem lag:

```
1. // src/lib/utils/pii-filter.ts
2. export function filterPII(
3.   text: string,
4.   userContext?: PiiFilterUserContext,
5. ): string {
6.   let filtered = text;
7.
8.   // 1. E-mailadresser
9.   filtered = filtered.replace(
10.    /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g,
11.    "[EMAIL REMOVED]",
12.  );
13.
14.   // 2. Danske CPR-numre (DDMMYY-XXXX eller DDMMYYXXXX)
15.   filtered = filtered.replace(/\\b\\d{6}-?\\d{4}\\b/g, "[CPR REMOVED]");
16.
17.   // 3. Telefonnumre (dansk format og +45-prefiks)
18.   filtered = filtered.replace(
19.    /(?:\\+45\\s*)?\\b\\d{2}\\s*\\d{2}\\s*\\d{2}\\s*\\d{2}\\b/g,
20.    "[PHONE REMOVED]",
21.  );
22.
23.   // 4. Brugerens eget navn (hvis tilgængeligt i kontekst)
24.   if (userContext) {
25.     const namesToRedact = [
26.       userContext.name,
27.       userContext.firstName,
28.       userContext.lastName,
29.       ...(userContext.name?.split(/\\s+/) ?? []),
30.     ].filter((n): n is string => typeof n === "string" && n.length > 2);
31.
32.     namesToRedact.forEach((name) => {
33.       const safeName = name.replace(/\\.[*+?^$\\{\\}\\|\\[\\]\\]/g, "\\$&");
34.       const regex = new RegExp(`\\b${safeName}\\b`, "gi");
35.       filtered = filtered.replace(regex, "[NAME REMOVED]");
36.     });
37.   }
38.
39.   // Rydningsop: slå sammenstødende navnetags sammen
40.   filtered = filtered.replace(
41.    /(\\[NAME REMOVED\\])\\s*\\[NAME REMOVED\\]+/g,
42.    "[NAME REMOVED]",
43.  );
44.
45.   // 5. Sikkerhedsrydning: indsæt mellemrum mellem alle sammenstødende removal-tags
46.   filtered = filtered.replace(
47.    /(\\[\\[A-Z ]+REMOVED\\])(?=\\[\\[A-Z ]+REMOVED\\])/g,
48.    "$1 ",
49.  );
50.
51.   return filtered;
52. }
```

Kodeudsnit 31: filterPII-funktionen i src/lib/utils/pii-filter.ts med de fem filterlag. Regex for CPR-numre (\\b\\d{6}-?\\d{4}\\b) matcher begge formater (med og uden bindestreg). Navnefilteret escape'r brugernavnet via replace(/\\.[+?^\$\\{\\}\\|\\[\\]\\]/g, "\\\$&") for at forhindre regex-injection fra brugerprofildata. Trin 5 er en sekundær sikkerhedsrydning der indsætter mellemrum mellem evt. sammenstødende removal-tags på tværs af alle tagtyper.*

`filterPII` filtrerer udelukkende mønstre der kan genkendes fra struktur (e-mail @-tegnet, CPR'ets 10-cifrede format, telefonnummerets 8-cifrede blokstruktur). Semantisk PII - adresser, fødselsdatoer, diagnoseoplysninger der skrives i naturligt sprog - identificeres ikke og sendes ufiltreret til Gemini. Dette er en bevidst og dokumenteret begrænsning: en fuldt semantisk PII-filter ville kræve en LLM, der i sig selv introducerer latens og omkostninger der overstiger filtrets nytteværdi for en MVP-implementering.

Geminis system prompt instruerer modellen om aldrig at afsløre over for brugeren, at PII er filtreret - dette er dokumenteret i `SYSTEM_PROMPT`-konstanten i `niqo.service.ts`:

```
1. // src/services/niqo.service.ts
2. const SYSTEM_PROMPT = `You are Niqo, an empathetic, understanding, and supportive AI companion on
Chroniqo, a platform for people living with chronic illnesses and fatigue.
3. Your goal is to listen, validate feelings, and offer gentle encouragement without being overly positive or toxic.
4. You must keep your responses concise (max 2-3 short paragraphs).
5. You do not give medical advice under any circumstances. If medical advice is requested, gently remind the user to consult a healthcare professional.
6. Never reveal that you are filtering PII.`;
7.
```

Kodeudsnit 32: `SYSTEM_PROMPT`-konstanten i `src/services/niqo.service.ts`, som definerer modellens persona, begrænsninger for svarlængde samt håndtering af medicinsk rådgivning og filtreret PII.

4.4.2. Redis TTL-baseret sessionshåndtering

Niqos samtalehistorik gemmes persisteret i PostgreSQL (NiqoChat og NiqoMessage-modellerne), men kontekstvinduet der sendes til Gemini er sessionsbegrænset: `generateResponse` og `sendMessage` konstruerer altid historikken fra databasen, ikke fra Redis.

Redis spiller en indirekte rolle via `rateLimiterPerHour`: brugerens Gemini-kvote spores som en sliding-window-tæller i Redis med én times TTL. Når en ny chat-session oprettes via `startNewChat`, oprettes en ny `NiqoChat`-række og det forrige chat-objekt slettes:

```
1. // src/services/niqo.service.ts
2. async startNewChat(userId: string, content: string): Promise<string> {
3.   const user = await prisma.user.findUnique({ where: { id: userId } });
4.   if (!user) throw new Error("User not found");
5.
6.   try {
7.     await prisma.niqoChat.delete({ where: { userId } });
8.   } catch { /* ingen tidligere chat at rydde op */ }
9.
10.  const chat = await prisma.niqoChat.create({ data: { userId } });
11.
12.  const filteredContent = filterPII(content, user);
13.  await prisma.niqoMessage.create({
14.    data: { niqoChatId: chat.id, role: "USER", content: filteredContent },
15.  });
16.
17.  return chat.id;
18. },
```

Kodeudsnit 33: `startNewChat`-funktionen i `src/services/niqo.service.ts`. Brugeren hentes og valideres med en `guard` inden videre eksekvering. `prisma.niqoChat.delete` er indkapslet i `try/catch` da `delete` kaster en `PrismaClientKnownRequestError` hvis rækken ikke eksisterer. `filterPII` kaldes med det fulde `User`-objekt som kontekst, så brugerens fornavn, efternavn og displaynavn maskeres i beskeden.

`NiqoChat` har en `@unique`-relation til `User` (`userId @unique`) der håndhæver, at en bruger maksimalt har én aktiv chat ad gangen. `startNewChat` sletter altid det eksisterende chat-objekt - og dermed al tilknyttet historik - inden en ny oprettes. Dette er det session-baserede aspekt: historikken varer kun så længe brugeren ikke starter en ny chat.

4.4.3. Server-side Gemini API-integration og to-trins-flow

`NiqoService` implementerer et to-trins-flow der adskiller chatoprettelse fra AI-svaret:

1. **Trin 1 - `startNewChat`:** Gemini kaldes ikke. Brugerens første besked gemmes i databasen, og chat-id'et returneres til klienten. Klienten omdirigeres til chat-siden med det nye id.
2. **Trin 2 - `generateResponse`:** Kaldes af `/api/niqo/{id}/generate-route-handler` efter at klienten er på chat-siden.

Gemini-kaldet sker her:

```
1. // src/services/nico.service.ts
2. async generateResponse(userId: string, chatId: string): Promise<string> {
3.   const user = await prisma.user.findUnique({ where: { id: userId } });
4.   if (!user) throw new Error("User not found");
5.
6.   const chat = await prisma.nicoChat.findUnique({
7.     where: { id: chatId },
8.     include: { messages: { orderBy: { createdAt: "asc" } } },
9.   });
10.
11.   if (!chat || chat.userId !== userId) {
12.     throw new Error("Chat not found or unauthorized");
13.   }
14.
15.   const lastMsg = chat.messages[chat.messages.length - 1];
16.   if (!lastMsg || lastMsg.role !== "USER") {
17.     throw new Error("No pending user message to respond to");
18.   }
19.
20.   // Alle beskeder undtagen den seneste udgør chat-historikken
21.   const history = chat.messages.slice(0, -1).map((msg) => ({
22.     role: msg.role === "MODEL" ? "model" : "user",
23.     parts: [
24.       {
25.         text:
26.           msg.role === "MODEL" ? msg.content : filterPII(msg.content, user),
27.       },
28.     ],
29.   }));
30.
31.   const filteredLastMsg = filterPII(lastMsg.content, user);
32.
33.   let aiResponseText =
34.     "I am having trouble connecting right now. Please try again in a moment.";
35.   try {
36.     const model = getGenerativeModel();
37.     const chatSession = model.startChat({ history });
38.     const result = await chatSession.sendMessage(filteredLastMsg);
39.     aiResponseText = result.response.text();
40.   } catch (error: unknown) {
41.     console.error(
42.       "[NicoService] Gemini API Error (generateResponse):",
43.       error instanceof Error ? error.message : error,
44.     );
45.   }
46.
47.   await prisma.nicoMessage.create({
48.     data: { nicoChatId: chatId, role: "MODEL", content: aiResponseText },
49.   });
50.
51.   return aiResponseText;
52. },
```

Kodeudsnit 34: `generateResponse`-funktionen i `src/services/nico.service.ts`. Brugerhentning og autorisatjonstjek (`chat.userId !== userId`) sikrer at en bruger ikke kan kalde andres chathistorik. `LastMsg`-validering forhindrer et Gemini-kald uden en ventende brugerbesked. Gemini-kaldet er indkapslet i `try/catch` med en fallback-svarstreng, så et midlertidigt API-nedbrud returnerer en meningsfuld fejlbesked frem for at crashe. `filterPII` anvendes på alle historik-beskeder fra brugeren, da det er ukendt om historikken indeholder PII der ikke var filtreret ved oprettelsestidspunktet.

Adskillelsen af de to trin er motiveret af brugeroplevelsen: hvis `startNewChat` afventede Gemini-svaret synkront (typisk 1-3 sekunders latens), ville ny-chat-dialogvinduet hænge i dette interval. To-trins-tilgangen lader dialogvinduet lukke øjeblikkeligt ved chat-oprettelse, mens klienten præsenteres for en loading-indikator på chat-siden mens `generateResponse` eksekveres asynkront.

`getGenerativeModel` instantierer `GoogleGenerativeAI`-klienten og konfigurerer `gemini-2.5-flash-lite`-modellen med `SYSTEM_PROMPT` som system-instruktion. Flash-Lite-varianten er valgt til Niquo fordi dens lave inferenslatens og høje token-throughput er velegnet til chattens hyppige, korte svar - til forskel fra `gemini-2.5-flash` der er optimeret til mere komplekse ræsonneringsopgaver (19).

4.5. Chat og short-polling - implementeringsdetaljer

Chat-modulet hviler på to sammenhængende mekanismer: polling-konfigurationen der driver den oplevede realtidsfølelse via SWR (afsnit 4.5.1), og samtaleudløbslogikken der håndhæver TTL-begrænsningen via et planlagt cron-job (afsnit 4.5.2).

4.5.1. SWR-baseret polling-konfiguration

Chat-funktionaliteten er implementeret med short-polling via SWR frem for WebSockets. Begrundelsen er arkitektonisk: Vercel-deployments kører som serverless functions, der instansieres og termineres per-request og dermed ikke kan opretholde en persistent TCP-forbindelse, som WebSockets kræver (20). Short-polling er det teknisk korrekte valg inden for serverless-begrænsningerne.

`src/lib/hooks/use-chat.ts` definerer samtlige polling-hooks med differentierede intervaller:

```
1. // src/lib/hooks/use-chat.ts
2. export function useConversations() {
3.   return useSWR("/api/conversations", fetcher, {
4.     refreshInterval: 10000, // 10 sekunder - samtalelisten
5.     revalidateOnFocus: true,
6.   });
7. }
8.
9. export function useMessages(conversationId: string | null) {
10.  return useSWR(
11.    conversationId ? `/api/conversations/${conversationId}/messages` : null,
12.    fetcher,
13.    {
14.      refreshInterval: 3000, // 3 sekunder - aktiv samtale
15.      revalidateOnFocus: true,
16.    },
17.  );
18. }
19.
20. export function useMinigame(gameId: string | null, mode: "ASYNC" | "LIVE" = "ASYNC") {
21.  return useSWR(gameId ? `/api/minigames/${gameId}` : null, fetcher, {
22.    refreshInterval: mode === "LIVE" ? 1500 : 5000, // 1,5s i LIVE, 5s i ASYNC
23.    revalidateOnFocus: true,
24.  });
25. }
26.
27. export function useConversationMinigames(conversationId: string | null) {
28.  return useSWR(
29.    conversationId ? `/api/conversations/${conversationId}/minigames` : null,
30.    fetcher,
31.    { refreshInterval: 3000, revalidateOnFocus: true },
32.  );
33. }
```

Kodeudsnit 35: De fire primære polling-hooks i `src/lib/hooks/use-chat.ts`. `useMinigame` illustrerer dynamisk intervalkonfiguration baseret på spilmode-parameteret: `LIVE`-mode halverer intervallet fra 5.000 ms til 1.500 ms for at reducere oplevet latens når begge spillere er online.

Polling-intervallerne er designet hierarkisk ud fra, hvor hurtigt indholdet ændrer sig:

Tabel 8: SWR polling-intervaller og revalideringsstrategier for chat-hooks i `src/Lib/hooks/use-chat.ts` - hook-navn, interval og begrundelse for det valgte interval.

SWR polling-intervaller og revalideringsstrategier for chat-hooks i use-chat.ts		
Hook	Interval	Begrundelse
<code>useConversations</code>	10.000 ms	Samtalelisten - nye samtaler opstår sjældent
<code>useMessages</code>	3.000 ms	Aktiv chat - tilstrækkelig til "near real-time"-oplevelse
<code>useNotesCarousel</code>	60.000 ms	Dagsformsoversigt - ændres maksimalt én gang dagligt
<code>useMinigame</code> (ASYNC)	5.000 ms	Asynkront spil - modpartens træk afventes uden hastende interval
<code>useMinigame</code> (LIVE)	1.500 ms	Live-spil - begge spillere er online, hyppigere polling ønskes
<code>useConversationMinigames</code>	3.000 ms	Spilinvitationer i samtale - samme kadence som beskeder

`revalidateOnFocus: true` betyder, at SWR automatisk genvaliderer cachen, når browservinduet fokuseres - en bruger der vender tilbage til chat-siden efter at have gjort noget andet, ser øjeblikkeligt de seneste beskeder uden at vente på næste polling-interval.

`null` som SWR-nøgle (f.eks. `conversationId ? url : null`) er SWRs konvention for betingede fetch-kald: hooks med `null`-nøgle eksekveres ikke, men returnerer `undefined` for `data` og `isLoading = false`. Dette bruges til at undgå at hente beskedhistorik, når ingen samtale er valgt (20).

4.5.2. Samtaleudløb og cron-job (/api/cron/prune-chats)

Samtaler har en konfigurerbar TTL styret af `Conversation.durationHours`-feltet (standard: 24 timer). `expiresAt`-tidsstemplet beregnes ved samtaleoprettelse som `now + durationHours * 3600 * 1000` og gemmes som et `DateTime` i databasen.

Udløbende samtaler slettes dagligt kl. 02:00 UTC af `/api/cron/prune-chats`-endpointet (konfigureret i `vercel.json`, se afsnit 5.2). Route Handleren udfører ét atomisk `deleteMany`-kald der sletter samtaler, som opfylder *enten* af to betingelser:

```
1. // src/app/api/cron/prune-chats/route.ts (skematisk)
2. const result = await prisma.conversation.deleteMany({
3.   where: {
4.     OR: [
5.       { expiresAt: { lte: new Date() } }, // TTL-udløb (inklusive)
6.       { deletionScheduledAt: { lte: new Date() } }, // Manuel sletnings-timer udløbet (inklusive)
7.     ],
8.   },
9. });
```

Kodeudsnit 36: Cron-logikken i `/api/cron/prune-chats`. Et enkelt `deleteMany` med OR-betingelse håndterer begge sletningsscenarier: TTL-udløb (`expiresAt ≤ now`) og manuel bruger-initieret sletning (`deletionScheduledAt ≤ now`). Operatoren `Lte` (less than or equal) sikrer at samtaler der præcis udløber på udførsels-tidspunktet medtages. `onDelete: Cascade` på `Message`-relationerne eliminerer behovet for at slette beskeder separat.

OR-betingelsen afspejler to adskilte sletningsscenarier. Det første er TTL-baseret udløb: samtalen er nået sin naturlige levetid. Det andet er brugerinitieret sletning: en bruger der manuelt sletter en samtale, sætter `deletionScheduledAt` til `now + 15 minutter` i stedet for at slette øjeblikkeligt - dette giver en kort grace-periode, der giver mulighed for en fortrydelsesmekanisme for brugeren, inden cron-jobbet permanent får endnu en samtale at skulle slette. `deletionScheduledAt` er dermed ikke en post-expiry forsinkelse men en selvstændig manuel sletnings-timer.

`onDelete: Cascade` på `Message`-modellen sikrer, at alle beskeder i en slettet samtale slettes atomisk af PostgreSQL - cron-jobbet behøver ikke at iterere over beskeder individuelt, og der opstår ingen forældreløse (orphaned) beskeder. Dette er et konkret eksempel på, at forretningsreglen om datatømnings ved samtalslut er implementeret i databaselaget (afsnit 3.1.2) frem for i applikationslaget.

Brugere kan forlænge en samtale manuelt via chat-interfacet, der kalder `/api/conversations/{id}/extend` og opdaterer `expiresAt`-feltet. Samtalevarighed er dermed ikke en uforanderlig TTL men en brugervendt indstilling der kan justeres inden udløb.

5. Automatiseret deployment - konfigurationsdetaljer

Dette afsnit dokumenterer den samlede deploymentstrategi for Chroniqo, herunder den automatiserede GitLab CI/CD-pipeline, Vercel-konfigurationen, Docker-image-strategien og håndteringen af miljøvariabler på tværs af udviklings- og produktionsmiljøet.

5.1. GitLab CI/CD-pipeline

Chroniqos CI/CD-pipeline er implementeret i `.gitlab-ci.yml` og eksekveres automatisk af GitLab CI ved hvert commit til repositoryet på `gitlab.au.dk` (21). Pipelinen er struktureret i fem sekventielle faser (stages), der tilsammen sikrer at kode, der når produktionsmiljøet, er sikkerhedsauditeret, lintet, typetjekket, testet og succesfuldt bygget. De to deploymentjobs - `docker_image_build` og `vercel_deploy` - eksekveres udelukkende ved merge til `production`-branchen.

```
1. # .gitlab-ci.yml
2. # Node 20 alpine er et lille og hurtigt image, velegnet til Next.js 16 (kræver Node 20.9+)
3. image: node:20-alpine
4.
5. stages:
6.   - install
7.   - check      # Kombinerer audit, lint og typetjek til parallel eksekvering
8.   - test
9.   - build
10.  - package
11.
12. # Global cache-konfiguration.
13. # Cachen er nøglesat på package-lock.json: ændres .lock-filen, ugyldiggøres cachen automatisk.
14. cache:
15.   key:
16.     files:
17.       - package-lock.json
18.   paths:
19.     - node_modules/
20.     - .next/cache/
21.
22. # Globalt before_script: installerer libc6-compat og openssl (kræves af Next.js/sharp),
23. # kører npm ci i offline-tilstand og genererer Prisma-klienten.
24. before_script:
25.   - apk add --no-cache libc6-compat openssl
26.   - npm ci --prefer-offline --no-audit
27.   - npx prisma generate
28.
29. # JOB 0: install_dependencies
30. # Sikrer en ren, populere node_modules-cache til efterfølgende jobs.
31. install_dependencies:
32.   stage: install
33.   script:
34.     - npm ci
35.   artifacts:
36.     paths:
37.       - node_modules/
38.     expire_in: 1 hour
39.
40. # JOB 1: security_audit
41. # Auditerer produktionsafhængigheder mod NPM advisory register.
42. # --audit-level=critical: kun kritiske sårbarheder fejler buildet.
43. # --omit=dev: devDependencies auditeres ikke (de deployes ikke til produktion).
44. security_audit:
45.   stage: check
46.   script:
47.     - npm audit --audit-level=critical --omit=dev
48.   allow_failure: false
49.
50. # JOB 2: linting
51. # Kører ESLint over hele kodebasen (konfiguration i eslint.config.mjs).
52. linting:
53.   stage: check
```

```

54. script:
55.   - npm run lint
56.
57. # JOB 3: type_checking
58. # Verificerer fuld TypeScript-typekorrekthed uden at generere outputfiler.
59. type_checking:
60.   stage: check
61.   script:
62.     - npx tsc --noEmit
63.
64. # JOB 4: unit_tests
65. # Kører Jest-testsuiten med --ci-flag (deaktiverer interaktive watch-modes).
66. # Genererer JUnit XML-rapport til GitLab CI's testrapporteringsvisning.
67. # when: always sikrer at testrapporten uploades selv ved fejlede tests.
68. unit_tests:
69.   stage: test
70.   script:
71.     - npm run test:ci
72.   artifacts:
73.     when: always
74.     reports:
75.       junit: junit.xml
76.
77. # JOB 5: production_build
78. # Kører Next.js produktionsbygning (next build). Validerer at applikationen
79. # kan kompileres til et deployerbart bundt. Outputet gemmes som artifact i
80. # én uge så package-fasens jobs kan anvende det præbyggede bundt.
81. production_build:
82.   stage: build
83.   script:
84.     - npm run build
85.   artifacts:
86.     paths:
87.       - .next/
88.       - public/
89.       - package.json
90.       - next.config.ts
91.     expire_in: 1 week
92.
93. # JOB 6: docker_image_build
94. # Bygger og pusher Docker-image til Docker Hub med to parallelle tags:
95. #   $CI_COMMIT_SHORT_SHA - præcis rollback til specifikt commit
96. #   :latest               - seneste produktionsklare build
97. #
98. # Anvender Docker-in-Docker (dind) som sidecar-service.
99. # Triggerregel: kun ved merge til production-branch.
100. docker_image_build:
101.   stage: package
102.   image: docker:24.0.5
103.   services:
104.     - docker:24.0.5-dind
105.   variables:
106.     DOCKER_TLS_CERTDIR: "/certs"
107.   before_script:
108.     - docker info
109.     - docker login -u $DOCKER_HUB_USER -p $DOCKER_HUB_TOKEN
110.   script:
111.     - docker build
112.       -t $DOCKER_HUB_USER/chroniqo-app:$CI_COMMIT_SHORT_SHA
113.       -t $DOCKER_HUB_USER/chroniqo-app:latest .
114.     - docker push $DOCKER_HUB_USER/chroniqo-app:$CI_COMMIT_SHORT_SHA
115.     - docker push $DOCKER_HUB_USER/chroniqo-app:latest
116.   needs:
117.     - production_build
118.   rules:
119.     - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
120.

```

```
121. # JOB 7: vercel_deploy
122. # Deployer det præbyggede bundt til Vercel (--prebuilt undgår dobbeltbygning).
123. # VERCEL_TOKEN er gemt som CI/CD-variabel i GitLab-projektets indstillinger.
124. # Triggerregel: kun ved merge til production-branch.
125. vercel_deploy:
126.   stage: package
127.   image: node:20-alpine
128.   before_script:
129.     - npm install -g vercel
130.   script:
131.     - vercel pull --yes --environment=production --token=$VERCEL_TOKEN
132.     - vercel build --prod --token=$VERCEL_TOKEN
133.     - vercel deploy --prebuilt --prod --token=$VERCEL_TOKEN
134.   needs:
135.     - production_build
136.   rules:
137.     - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Kodeudsnit 37: Den komplette `.gitlab-ci.yml` - jobs 0-7 med inline-kommentarer til hvert jobs ansvar, triggerregel og afhængigheder. `needs`-direktivet på jobs 6 og 7 sikrer, at deployment kun initieres ved et succesfuldt `production_build`. Begge `package-jobs` kører parallelt og affyres udelukkende ved merge til `production-branchen`.

De tre jobs i `check-fasen` (`security_audit`, `linting`, `type_checking`) kører parallelt, hvilket minimerer den samlede pipeline-varighed. De to jobs i `package-fasen` (`docker_image_build` og `vercel_deploy`) eksekveres parallelt ved merge til `production-branchen` og er begge afhængige af et succesfuldt `production_build-job` via `needs`-direktivet.

5.2. Vercel-deployment - konfiguration og miljøvariabler

Produktionsdeploymentet hostes på Vercel. Al platform-specifik konfiguration er defineret i `vercel.json` i projektets rod; øvrig Vercel-konfiguration (build command, output directory, Node-version) håndteres automatisk af Vercels Next.js-preset og kræver ingen eksplicit angivelse.

5.2.1. vercel.json

```
1. {  
2.   "crons": [  
3.     {  
4.       "path": "/api/cron/prune-chats",  
5.       "schedule": "0 2 * * *"  
6.     },  
7.     {  
8.       "path": "/api/cron/prune-unverified-users",  
9.       "schedule": "0 3 * * *"  
10.    }  
11.  ]  
12. }
```

Kodeudsnit 38: Den komplette `vercel.json` i projektets rod. Filen konfigurerer udelukkende to cron-jobs med schedule i cron-notation (UTC). Al øvrig Vercel-konfiguration (build command, output directory, Node-version) håndteres automatisk af Vercels Next.js-preset.

`vercel.json` konfigurerer udelukkende to cron-jobs, der eksekveres dagligt af Vercels serverless scheduler. Schedules er angivet i cron-notation (UTC):

Tabel 9: Oversigt over projektets cron-jobs konfigureret i `vercel.json` - rute, schedule i cron-notation (UTC) og formålet med den daglige oprydningsoperation.

Oversigt over projektets cron-jobs samt deres formål			
Cron-job	Rute	Schedule	Formål
Samtaleudløb	<code>/api/cron/prune-chats</code>	<code>0 2 * * *</code> (kl. 02:00 UTC)	Sletter samtaler, hvor <code>expiresAt</code> -tidsstempet i <code>Conversation-model</code> len er overskredet. Realiserer den konfigurerbare chat-TTL (standard: 24 timer).
Ubekræftede brugere	<code>/api/cron/prune-unverified-users</code>	<code>0 3 * * *</code> (kl. 03:00 UTC)	Sletter brugerkonti, der ikke har fuldført e-mailbekræftelsen inden for det konfigurerede vindue. Forhindrer akkumulering af ufuldstændige konti i databasen.

Begge cron-ruter er beskyttet via en `CRON_SECRET`-header, der verificeres i den respektive Route Handler, inden databaseoperationen påbegyndes. Dette forhindrer uautoriseret ekstern aktivering af oprydningslogikken.

5.2.2. Miljøvariabler i Vercel-projektet

Nedenstående tabel viser samtlige miljøvariabler, der er konfigureret i Vercel-projektets produktionsmiljø. Variabelværdier er som altid udeladt af sikkerhedsmæssige hensyn.

Tabel 10: Samtlige miljøvariabler konfigureret i Vercel-projektets produktionsmiljø - variabelnavn, kategori og formål. Variabelværdier er udeladt af sikkerhedsmæssige hensyn.

Oversigt over miljøvariabler i Vercel-projektet		
Variabel	Kategori	Formål
NEXTAUTH_URL	Auth	Basisadressen for Auth.js' autentificeringsruter og callbacks
AUTH_SECRET	Auth	Krypteringsnøgle til JWT-sessionscookie - genereret med <code>npx auth secret</code>
GOOGLE_CLIENT_ID	OAuth	Client ID fra Google Cloud Console til Google OAuth 2.0 PKCE-login
GOOGLE_CLIENT_SECRET	OAuth	Client Secret til Google OAuth 2.0-flow
DATABASE_URL	Database	Neon.tech PostgreSQL-forbindelsestreng inkl. connection pooler-endpoint og SSL-parametre
EMAIL_USER	E-mail	Gmail-afsenderadresse til transaktionelle e-mails (onboarding, e-mailbekræftelse)
EMAIL_PASS	E-mail	Gmail App Password til Nodemailer SMTP-afsendelse
GEMINI_API_KEY	AI	API-nøgle til Google Gemini API (Niqo AI-støttebot)
UPSTASH_REDIS_REST_URL	Cache	REST-endpoint til Upstash Serverless Redis (rate limiting og ban-flag)
UPSTASH_REDIS_REST_TOKEN	Cache	Autentificeringstoken til Upstash Redis
BLOB_READ_WRITE_TOKEN	Storage	Adgangstoken til Vercel Blob Storage (medieupload: profilbilleder, opslag)
CRON_SECRET	Cron	Hemmelighed (Secret) der verificeres i cron Route Handlers for at forhindre uautoriseret ekstern aktivering. Kun konfigureret i Vercel-produktionsmiljøet - ikke anvendt lokalt, da cron-jobs ikke afvikles under lokal udvikling.

5.3. Docker-image og Container Registry

Docker anvendes udelukkende som pakkeformat for det produktionsklare applikationsimage. Lokal udvikling er foregået direkte på værtsmaskinens Node.js-installation med `npm run dev` og uden Docker Desktop - al interaktion med cloud-tjenesterne (Neon.tech, Upstash, Vercel Blob) sker via de konfigurerede miljøvariabler i `.env.local`, og der er ikke afviklet lokale container-instanser af hverken databasen eller applikationen.

5.3.1. Produktionsimage og Docker Hub

Produktionsimage bygges via `docker_image_build`-jobbet (jf. afsnit 5.1) og pushes til Docker Hub frem for GitLab Container Registry (22).

Årsagen er institutionel: projektet er versionsstyret på `gitlab.au.dk`, Aarhus Universitets eget GitLab-installation. GitLabs Container Registry kræver aktivering på instansniveau af en GitLab-administrator - denne konfiguration administreres af AU's IT-organisation og er ikke tilgængelig for projektejere på instansen. Et forsøg på at pushe til `registry.gitlab.au.dk` returnerer en autorisationsfejl trods korrekt konfigurerede projekt-credentials. Docker Hub er valgt som alternativ, da det ikke kræver instansniveaunkonfiguration og understøtter det nødvendige push-flow direkte via CI/CD pipelinens secrets (`DOCKER_HUB_USER` og `DOCKER_HUB_TOKEN`).

Images tagges med to parallelle tags ved hvert succesfuldt build til `production`:

- `:$CI_COMMIT_SHORT_SHA` - det korte Git commit-hash, der muliggør præcis rollback til et specifikt commit.
- `:latest` - det seneste produktionsklare image, der altid peger på det nyeste succesfulde build fra `production`-branchen.

5.4. Miljøvariabel-strategi - udviklings- vs. produktionsmiljø

I dette projekt er der ikke etableret en formel adskillelse mellem produktions- og udviklingsmiljøets konfiguration. Næsten samtlige miljøvariabler er identiske på tværs af det lokale udviklingsmiljø (`.env.local`) og Vercel-projektets produktionskonfiguration - det vil sige, at lokale kørsler med `npm run dev` tilgår de samme cloud-tjenester (Neon.tech, Upstash, Vercel Blob) som det deployerede produktionssystem.

Den eneste undtagelse er `CRON_SECRET`, der udelukkende er konfigureret i Vercel-projektets produktionsmiljø. Cron-jobs aktiveres af Vercels serverless scheduler og eksekveres ikke under lokal udvikling, hvorfor variabelen ikke er nødvendig i `.env.local`.

Dette er et bevidst valg truffet på baggrund af projektets optræden som soloprojekt i MVP-fasen. Med én udvikler og ingen parallelle teams eliminerer en delt konfiguration den operationelle overhead, der følger med adskilte miljøer: der er ingen risiko for at produktionsdata forveksles og/eller overskrives af testdata fra en anden udvikler, og konsistensen i konfigurationen fjerner en hel kategori af fejl der opstår ved miljøspecifik adfærd. Den praktiske gevinst er direkte målbar: en fejl der opstår på det deployerede system kan reproducere lokalt med nul konfigurationsændringer og afvikles i samme datalag.

Trade-off'et er velkendt og identificeret som et klart forbedringspunkt ved overgang fra MVP til teamudviklingsfase. Den korrekte praksis for et system med sensitive brugerdata er at etablere et dedikeret staging-miljø: separat Neon.tech-databaseinstans til testdata, separate Google OAuth 2.0-credentials i Cloud Console og et isoleret Upstash Redis-miljø, så udviklingsaktivitet ikke kan observere eller påvirke produktionsdata. Denne opsætning er trivielt at etablere i Vercel via projektets miljøkonfiguration, der understøtter separate variablesæt for `development`, `preview` og `production`.

6. Samlet konklusion

Dette bilag har dokumenteret implementeringsdetaljerne bag Chroniqos fire hovedlag med udgangspunkt i konkrete kodeudsnit, konfigurationsfiler og tekniske begrundelser.

På UI/UX-laget er kognitiv skånsomhed realiseret via et tokenbaseret designsystem i `globals.css`, der sikrer visuel konsistens på tværs af *Light-* og *Dark Mode* uden ændringer i komponentkoden. Dagsformssystemets tre lag - `DailyStatusContext`, `DailyStatusInterceptor` og `DailyStatusSlider` - samarbejder om at gøre registreringsflowet let og intuitivt, mens *Low Energy Mode* reducerer den kognitive belastning automatisk ved lave energiniveauer. FOUC-forebyggelsen via inline `themeScript` og `moodScript` sikrer, at brugeren aldrig oplever et visuelt flicker ved sideindlæsning.

I databaselaget er forretningsreglerne håndhævet direkte i Prisma-skemaet frem for i applikationslaget: `@@unique([userId, date])` håndhæver én dagsformsregistrering per bruger per kalenderdag, `onDelete: Cascade` realiserer GDPR-kravet om fuldstændig datosletning ved kontosletning, og sammensatte primærnøgler på koblingstabeller eliminerer dubletregistreringer og fungerer som implícite databaseindeks. `GlobalBan`-modellens `onDelete: SetNull`-semantik forhindrer omgåelse af bandlysninger via kontosletning.

Softwarearkitekturen følger konsekvent Separation of Concerns: Route Handlers er tynde controllers, der udelukkende autentificerer, validerer og delegerer, mens service-laget indkapsler al forretningslogik og dermed muliggør isoleret enhedstest. DTO-laget med Zod-skemaer sikrer over-posting-beskyttelse og end-to-end typesikkerhed fra HTTP-krop til Prisma-kald. `Auth.js`-konfigurationens `signIn`-callback fungerer som central adgangsvagt og samler ban-tjek, OAuth-verifikation og JWT-tilpasning med applikationsspecifikke felter i én operation. Redis-ban-mekanismen via `setBanFlag` og `clearBanFlag` adresserer derudover JWT-revokationsproblemet uden at belaste auth-flowet for ikke-bandlyste brugere.

Deploymentlaget er fuldt automatiseret via en fem-trins GitLab CI/CD-pipeline, der sikrer at kode der når produktionsmiljøet er sikkerhedsauditeret, lintet, typetjekket, testet og succesfuldt bygget. Vercel-konfigurationen er bevidst minimal og overlader al platformspecifik logik til to cron-jobs, der dagligt oprydder udløbende samtaler og ubekræftede brugerkonti.

7. Referencer

5. Tailwind Labs. Theme Variables - Tailwind CSS [Internet]. [år ukendt]. [hentet 14. marts 2026]. Tilgængelig fra: <https://tailwindcss.com/docs/theme>.
6. Vercel. Font Optimization - Next.js [Internet]. [år ukendt]. [hentet 15. marts 2026]. Tilgængelig fra: <https://nextjs.org/docs/app/getting-started/fonts>.
7. Mozilla Contributors. prefers-color-scheme - CSS | MDN [Internet]. [år ukendt]. [hentet 15. marts 2026]. Tilgængelig fra: <https://developer.mozilla.org/en-US/docs/Web/CSS/@media/prefers-color-scheme>.
8. WorkOS. Radix UI - Accessible, unstyled UI components for React [Internet]. [år ukendt]. [hentet 17. marts 2026]. Tilgængelig fra: <https://www.radix-ui.com/>.
9. Mozilla Contributors. Pointer events - Web APIs | MDN [Internet]. [år ukendt]. [hentet 18. marts 2026]. Tilgængelig fra: https://developer.mozilla.org/en-US/docs/Web/API/Pointer_events.
10. Ivanov P. react-virtuoso - Virtual scrolling component for React [Internet]. [år ukendt]. [hentet 19. marts 2026]. Tilgængelig fra: <https://virtuoso.dev/>.
11. Prisma Data Inc. Native database types - Prisma ORM Documentation [Internet]. [år ukendt]. [hentet 21. marts 2026]. Tilgængelig fra: <https://www.prisma.io/docs/orm/prisma-migrate/workflows/native-database-types>.
13. Europa-Parlamentet og Rådet for Den Europæiske Union. Europa-Parlamentets og Rådets forordning (EU) 2016/679 af 27. april 2016 (databeskyttelsesforordningen) [Internet]. 2016. [hentet 21. marts 2026]. Tilgængelig fra: <https://eur-lex.europa.eu/legal-content/DA/TXT/?uri=CELEX%3A32016R0679>.
14. Prisma Data Inc. Prototyping your schema with db push - Prisma ORM Documentation [Internet]. [år ukendt]. [hentet 22. marts 2026]. Tilgængelig fra: <https://www.prisma.io/docs/orm/prisma-migrate/workflows/prototyping-your-schema>.
15. Neon. Connection pooling - Neon Documentation [Internet]. [år ukendt]. [hentet 22. marts 2026]. Tilgængelig fra: <https://neon.com/docs/connect/connection-pooling>.
16. Auth.js contributors. Session strategies - Auth.js [Internet]. [år ukendt]. [hentet 24. marts 2026]. Tilgængelig fra: <https://authjs.dev/concepts/session-strategies>.
17. The OWASP Foundation. HttpOnly | OWASP Foundation [Internet]. [år ukendt]. [hentet 24. marts 2026]. Tilgængelig fra: <https://owasp.org/www-community/HttpOnly>.
18. Upstash Inc. Rate Limiting Algorithms - Upstash Ratelimit [Internet]. [år ukendt]. [hentet 26. marts 2026]. Tilgængelig fra: <https://upstash.com/docs/oss/sdks/ts/ratelimit/algorithms>.
19. Google LLC. Gemini models - Google AI for Developers [Internet]. [år ukendt]. [hentet 27. marts 2026]. Tilgængelig fra: <https://ai.google.dev/gemini-api/docs/models>.

20. Vercel. SWR - React Hooks for Data Fetching [Internet]. [år ukendt]. [hentet 28. marts 2026]. Tilgængelig fra: <https://swr.vercel.app/>.
21. GitLab Inc. GitLab CI/CD - GitLab Documentation [Internet]. [år ukendt]. [hentet 29. marts 2026]. Tilgængelig fra: <https://docs.gitlab.com/ee/ci/>.
22. Docker Inc. Docker Hub - Container Image Library [Internet]. [år ukendt]. [hentet 31. marts 2026]. Tilgængelig fra: <https://hub.docker.com/>.

8. Bilag

1. Villadsen MD. Chroniqo: Kildekode. Bilag 9. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
2. Villadsen MD. Chroniqo: Systemdesign & Arkitektur. Bilag 5. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
3. Villadsen MD. Chroniqo: Test & Validering. Bilag 7. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
4. Villadsen MD. Chroniqo: UI-Dokumentation. Bilag 8. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
12. Villadsen MD. Chroniqo: Kravspecifikation. Bilag 1. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.